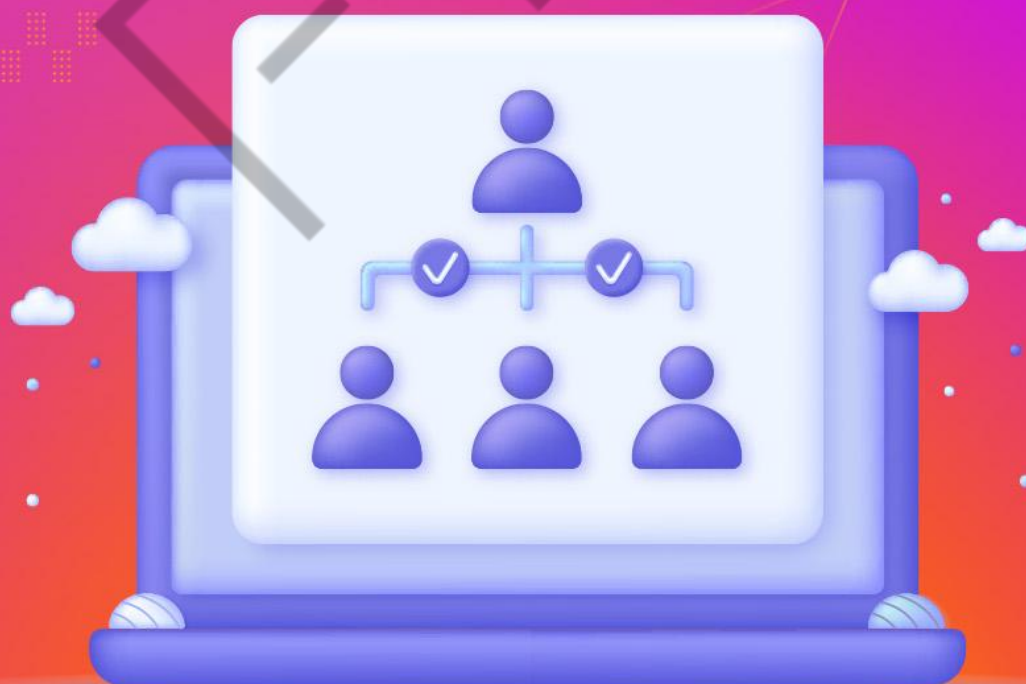


GESTÃO DE CLUSTERS,
SEGURANÇA QUÂNTICA E
COLABORAÇÃO MULTIAGENTE

ORQUESTRANDO O PODER: CONSTRUÇÃO, GERENCIAMENTO E MONITORAMENTO DE CLUSTERS



LISTA DE FIGURAS

Figura 1 - Topologia lógica mínima de um cluster	7
Figura 2 - Separação de redes para gestão e dados	8
Figura 3 - Alternativas de armazenamento e trade-offs	9
Figura 4 - Cadeia de gargalos: alinhando capacidades ao fluxo	13
Figura 5 - Failover com IP Virtual (Virtual IP – VIP)	16
Figura 6 - Quórum e risco de split-brain em partição de rede	17
Figura 7 - Remoção de SPOF com redundância N+1	18
Figura 8 - Arquitetura web com IP Virtual e nó de reserva (modelo de Alta Disponibilidade)	21
Figura 9 - Arquitetura de Alta Disponibilidade para repositório de dados crítico com replicação controlada	22
Figura 10 - L4 <i>versus</i> L7: onde o balanceamento decide.....	24
Figura 11 - Estratégia de balanceamento: decisão por características do tráfego	26
Figura 12 - Curva conceitual de latência <i>versus</i> carga.....	29
Figura 13 - Pipeline de cache: reduzindo pressão no backend	31
Figura 14 - Arquitetura de escalabilidade horizontal com balanceador e múltiplas instâncias (modelo inspirado em AWS).....	33
Figura 15 - Organização de acesso a dados em laboratório acadêmico com múltiplas turmas	34
Figura 16 - Arquitetura mínima de observabilidade	38
Figura 17 - Ciclo de diagnóstico de desempenho	39
Figura 18 - Dashboard de saúde do cluster (modelo conceitual)	42
Figura 19 - Defesa em profundidade: camadas de segurança.....	43
Figura 20 - Painel mínimo de monitoramento por nó	46
Figura 21 - Camadas básicas de reforço de segurança e auditoria em cluster corporativo.....	47

LISTA DE QUADROS

Quadro 1 - Matriz de decisão (exemplo didático) para escolhas de arquitetura.....	9
Quadro 2 - Exemplo de lista de componentes para laboratório.....	11
Quadro 3 - Padrões de HA e quando usar (visão qualitativa)	19
Quadro 4 - Algoritmos comuns de balanceamento (visão aplicada)	28
Quadro 5 - Alavancas práticas de eficiência em clusters (qualitativas)	35
Quadro 6 - Métricas essenciais para gestão de desempenho (visão geral)	41
Quadro 7 - RPO e RTO (interpretação prática)	45
Quadro 8 - Controles básicos de segurança (checklist orientado a prática)	48

SUMÁRIO

1 ORQUESTRANDO O PODER: CONSTRUÇÃO, GERENCIAMENTO E MONITORAMENTO DE CLUSTERS	5
1.1 Planejamento e Arquitetura de um Cluster	6
1.1.1 Critérios de decisão arquitetural	6
1.2 Crescimento e limites arquiteturais	11
2 GERENCIAMENTO DE CLUSTERS DE ALTA DISPONIBILIDADE	15
2.1 Identificação de pontos críticos e failover	15
2.2 Coordenação, quórum e modelos de redundância	16
3 BALANCEAMENTO DE CARGA E EFICIÊNCIA	24
3.1 Estratégias de distribuição de carga	24
3.2 Eficiência operacional e gestão de capacidade	27
4 GERENCIAMENTO DE DESEMPENHO E SEGURANÇA EM CLUSTERS	37
4.1 Observabilidade e diagnóstico de desempenho	37
4.2 Segurança operacional e proteção de dados	44
REFERÊNCIAS	49

1 ORQUESTRANDO O PODER: CONSTRUÇÃO, GERENCIAMENTO E MONITORAMENTO DE CLUSTERS

A consolidação de ambientes distribuídos transformou a forma como serviços digitais, pipelines de dados e aplicações de Inteligência Artificial (IA) são sustentados em produção. Em vez de depender de uma única máquina “forte”, organizações modernas buscam compor capacidade, resiliência e previsibilidade a partir de múltiplos recursos coordenados. Nesse contexto, um cluster é menos um conjunto de computadores e mais uma infraestrutura integrada, na qual conectividade, armazenamento e controle operacional precisam trabalhar como um único sistema para entregar resultados contínuos.

Este capítulo trata da implementação e do gerenciamento dessa infraestrutura sob uma ótica de projeto e operação. A ênfase está em como estruturar componentes e responsabilidades — quem decide, quem executa, quem armazena e por onde os dados circulam — para que o ambiente seja escalável sem se tornar frágil. A construção de um cluster envolve escolhas que definem limites de desempenho e de confiabilidade desde o início: uma rede mal segmentada amplia riscos e instabilidades; um armazenamento inadequado cria gargalos de Entrada/Saída (Input/Output – I/O); e um desenho sem redundância transforma falhas comuns em indisponibilidades.

O ponto central é a coerência arquitetural: decisões “locais” afetam o sistema inteiro. Uma topologia de rede pode favorecer throughput (vazão) e, ao mesmo tempo, dificultar isolamento; redundância pode aumentar disponibilidade, mas elevar a complexidade de consenso e recuperação; balanceamento de carga melhora a utilização, mas exige critérios de roteamento e verificação de saúde. Em ambientes reais, a solução mais eficiente não é a que maximiza um único indicador, e sim a que mantém comportamento estável sob variações de carga, falhas parciais e crescimento incremental — com governança operacional compatível com o time que irá manter o ambiente.

Ao longo das seções, será apresentado um encadeamento lógico de decisões: primeiro, o planejamento da arquitetura e da capacidade; em seguida, os mecanismos de Alta Disponibilidade (High Availability – HA) para reduzir interrupções; depois,

estratégias de balanceamento de carga e eficiência para manter desempenho sob demanda; e, por fim, práticas de observabilidade e segurança para controlar riscos, diagnosticar gargalos e garantir continuidade. O objetivo é consolidar uma visão prática e consistente de como desenhar e operar um cluster como plataforma confiável para serviços, dados e IA em cenários atuais de mercado.

1.1 Planejamento e Arquitetura de um Cluster

Planejar um cluster é escolher, antecipadamente, onde o sistema deve ser simples e onde ele precisa ser robusto. Para um primeiro cluster acadêmico, o objetivo costuma ser previsibilidade: garantir que vários grupos consigam utilizar o ambiente sem que uma atividade de um grupo “derrube” o trabalho de outro.

Em um ambiente corporativo, o objetivo tende a incluir níveis de serviço (Service Level Agreement – SLA), auditoria e capacidade de evoluir sem interrupções grandes. A mesma arquitetura pode servir aos dois casos, mas as prioridades mudam.

1.1.1 Critérios de decisão arquitetural

O ponto de partida é sempre a carga de trabalho (workload). Em um curso de Tecnologia em Inteligência Artificial, é comum haver três perfis: (i) tarefas de treinamento e validação de modelos, com uso intenso de Unidade de Processamento Gráfico (Graphics Processing Unit – GPU) quando disponível; (ii) tarefas de processamento de dados, com uso pesado de entrada e saída (Input/Output – I/O) e rede; e (iii) tarefas de serviço, como notebooks e APIs de inferência, onde latência e disponibilidade importam mais do que pico de desempenho. Planejar o cluster é mapear qual perfil é dominante e quais são os piores casos (picos) que precisam ser suportados.

Em seguida, definem-se os papéis dos nós. A prática mais comum separa pelo menos três grupos: nó de controle (control plane), nós de computação e nós/serviços de armazenamento. Em clusters pequenos, esses papéis podem coexistir na mesma máquina, mas isso aumenta o risco de interferência e dificulta diagnósticos. Em clusters médios, separar papéis melhora o isolamento: um problema de

armazenamento, por exemplo, não “contamina” o nó que faz o agendamento e o gerenciamento do ambiente.

A figura “Topologia lógica mínima de um cluster” mostra uma organização típica: usuários acessam o cluster por um ponto controlado (login/API), o nó de controle orquestra o uso de recursos, e a rede interna conecta nós de computação e armazenamento compartilhado. O leitor deve notar que a comunicação interna do cluster é distinta do tráfego externo, o que ajuda a manter previsibilidade e segurança.

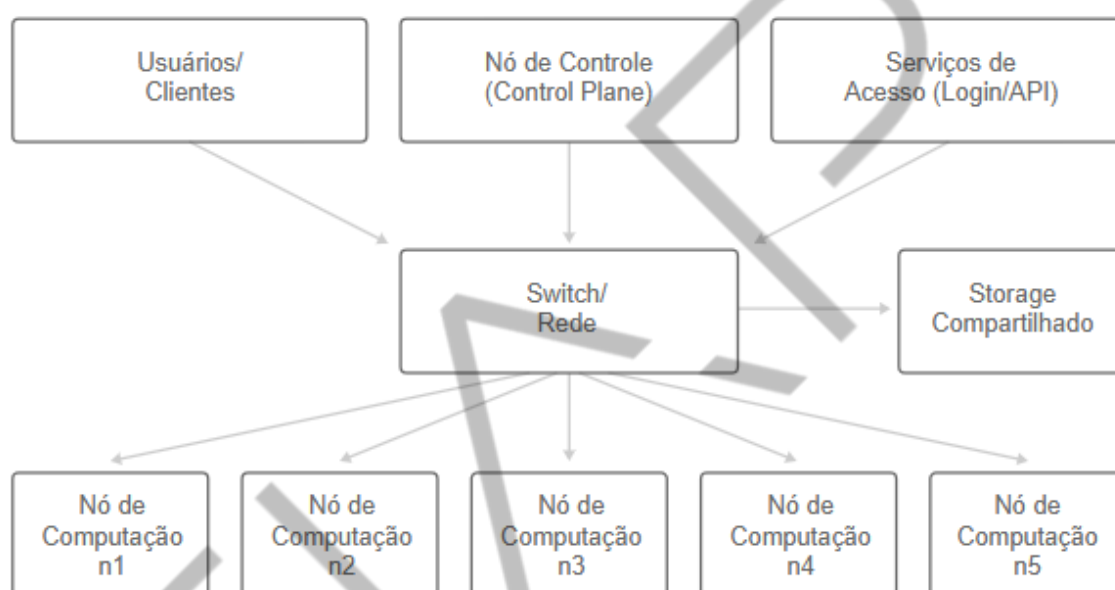


Figura 1 - Topologia lógica mínima de um cluster
Fonte: Elaborado pelo autor (2026)

A arquitetura física vem logo depois: quantos nós, quais capacidades e como conectá-los. Em termos didáticos, é útil pensar no “triângulo de capacidade”, processamento (Central Processing Unit – CPU), memória (Random Access Memory – RAM) e rede/armazenamento (I/O). Um erro comum em planejamentos iniciais é superestimar CPU e subestimar I/O. Em tarefas de IA, um conjunto de dados grande pode transformar o disco em gargalo; em tarefas distribuídas, a rede pode virar o limite. Assim, um cluster equilibrado não é o que tem mais CPU, mas o que tem capacidades compatíveis com o caminho real dos dados.

A figura a seguir ilustra a separação entre uma rede de gerenciamento (para acesso administrativo e operação) e uma rede de dados (para tráfego pesado de aplicações). O leitor deve observar que essa separação simplifica políticas de segurança e reduz a interferência entre operações administrativas e o tráfego de aplicações.

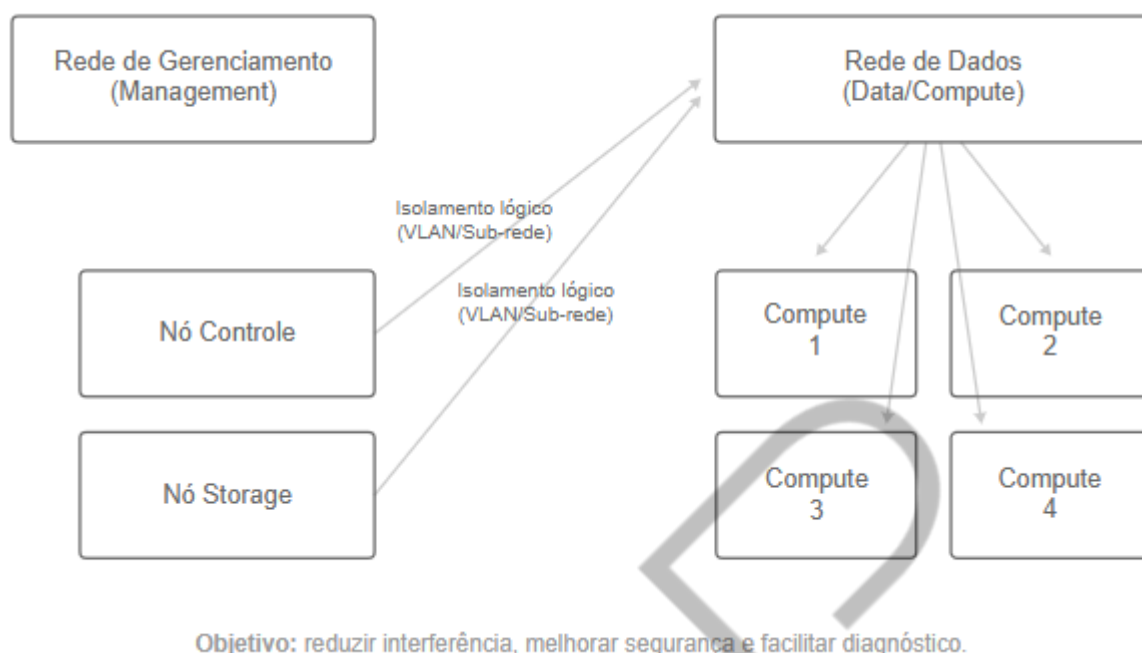


Figura 2 - Separação de redes para gestão e dados
Fonte: Elaborado pelo autor (2026)

No planejamento de rede, além do tipo de interconexão, a topologia física do laboratório (rack, sala, cabeamento) e o orçamento ditam o que é possível. Em geral, a recomendação é: comece com uma rede Ethernet bem dimensionada, com switches adequados e possibilidade de crescimento, e evolua para interconexões de maior desempenho quando houver necessidade comprovada. A compra de interconexão “de ponta” sem demanda real tende a consumir orçamento que poderia melhorar memória, armazenamento ou redundância, itens que frequentemente trazem ganhos mais visíveis no uso cotidiano.

Armazenamento é o outro eixo crítico. A escolha entre um servidor simples de compartilhamento, uma rede de armazenamento (Storage Area Network – SAN) ou um armazenamento distribuído afeta custo, desempenho e disponibilidade. Para turmas e laboratórios, um servidor de arquivos centralizado pode ser suficiente, desde que exista um plano de backup e que o risco de ponto único de falha (Single Point of Failure – SPOF) seja conhecido. À medida que o cluster cresce, soluções distribuídas se tornam atraentes por oferecerem escala e resiliência, porém exigem operação mais cuidadosa e monitoramento contínuo.

A figura “Alternativas de armazenamento e trade-offs” resume três caminhos comuns: compartilhamento simples (exemplo: Network File System – NFS), SAN e armazenamento distribuído (exemplo: Ceph). O leitor deve notar que há um equilíbrio

entre simplicidade operacional e robustez/escala. Em ambientes pequenos, a simplicidade tende a vencer; em ambientes críticos, a resiliência passa a ser prioridade.



Figura 3 - Alternativas de armazenamento e trade-offs
Fonte: Elaborado pelo autor (2026)

Um modo prático de planejar é construir uma matriz de decisão baseada em critérios. Três critérios são especialmente úteis: custo total de propriedade (Total Cost of Ownership – TCO), desempenho percebido pelo usuário e capacidade de operar/manter com o time disponível. Um cluster que exige especialistas para manter cada ajuste fino pode ser “tecnicamente excelente”, mas inadequado para uma organização com equipe pequena. Ao contrário, um cluster simples e bem monitorado frequentemente entrega melhor valor ao longo do tempo.

Decisão	Opção A (simples)	Opção B (intermediária)	Opção C (avançada)
Rede	Ethernet única	Ethernet separada (mgmt/dados)	Rede dedicada de alto desempenho + mgmt
Storage	NFS central	NFS + redundância/backup forte	Distribuído (exemplo: Ceph)
Controle	1 nó controle	Controle + nó de acesso separado	Controle redundante + automação
Operação	Baixa complexidade	Média	Alta (exige processos)

Quadro 1 - Matriz de decisão (exemplo didático) para escolhas de arquitetura
Fonte: Elaborado pelo autor (2026)

Como síntese desse raciocínio, o quadro “Matriz de decisão (exemplo didático) para escolhas de arquitetura” organiza escolhas típicas de arquitetura em três níveis de maturidade (opção simples, intermediária e avançada), permitindo visualizar

rapidamente o que se ganha e o que se “paga” (em custo e complexidade) a cada evolução.

Na linha **Rede**, “Ethernet” representa a rede padrão utilizada na maioria dos ambientes; a opção intermediária propõe separar tráfego de mgmt/dados, isto é, management (gerenciamento) e dados: uma rede (ou segmentação lógica) fica dedicada à administração e monitoramento, enquanto outra fica focada no tráfego das aplicações, reduzindo interferência e facilitando segurança e diagnóstico. Já a opção avançada adiciona uma rede dedicada de alto desempenho (com maior vazão e menor latência) mantendo uma rede de gerenciamento em paralelo, o que tende a beneficiar cargas intensas de comunicação, mas aumenta custo e requisitos de operação.

Na linha **Storage**, o quadro compara formas de armazenamento compartilhado. A sigla NFS significa Network File System, um modelo em que um servidor central oferece pastas compartilhadas para os nós do cluster. Por ser simples e barato, aparece como opção inicial (“NFS central”), mas pode se tornar ponto único de falha e de gargalo.

Na opção intermediária, mantém-se NFS, porém com redundância e backup forte (por exemplo, cópias regulares, espelhamento e plano de restauração), o que melhora a continuidade do serviço. Na opção avançada, o armazenamento passa a ser distribuído, com exemplo do Ceph (um sistema de storage distribuído), que favorece escalabilidade e resiliência, porém exige mais cuidado operacional e observabilidade.

Na linha **Controle**, “1nó controle” indica um único servidor responsável por coordenar o cluster (agendamento, serviços centrais, autenticação e afins). A opção intermediária separa o controle de um nó de acesso (por exemplo, um nó de login/API), o que melhora a organização e reduz o risco de sobrecarga no controle. A opção avançada propõe controle redundante + automação, isto é, mais de um nó capaz de assumir a função de controle em caso de falha e com procedimentos automatizados para reduzir intervenção manual — aumentando confiabilidade, mas exigindo processos mais bem definidos.

Por fim, a linha **Operação** resume o “custo invisível” dessas escolhas: baixa complexidade (mais fácil de manter, menos dependência de especialistas), média

(exige rotinas e monitoramento consistentes) e alta (necessita processos formais, documentação, testes de failover/recuperação e uma cultura operacional mais madura). Em outras palavras, o quadro anterior ajuda a decidir até onde sofisticar a arquitetura sem criar um sistema bom no papel, mas difícil de sustentar na prática.

1.2 Crescimento e limites arquiteturais

Uma vez definida a lógica de decisão (critérios e *trade-offs*) e visualizadas as alternativas na matriz, o próximo passo é traduzir essa escolha em uma configuração concreta. Em outras palavras: qual conjunto mínimo de componentes materializa a opção selecionada para um cenário realista? Para tornar essa passagem mais clara, o exemplo a seguir descreve um caso típico de laboratório acadêmico, no qual o objetivo é manter o ambiente previsível, fácil de operar e com capacidade de crescimento por adição de nós.

- **Exemplo prático 1 (laboratório acadêmico, 10–30 nós):** imagine uma disciplina com três turmas utilizando o cluster em horários diferentes. O desenho prioriza isolamento e previsibilidade. Uma solução típica é adotar um nó de controle e acesso (login), um servidor de armazenamento central com backups frequentes e um conjunto de nós homogêneos de computação. A rede pode ser segmentada logicamente para separar administração e dados. O resultado esperado é um ambiente que “não surpreende”: os alunos aprendem a submeter tarefas e interpretar resultados, e o professor mantém controle do uso.

No quadro “Exemplo de lista de componentes para laboratório”, essa arquitetura é traduzida em uma lista objetiva de componentes (Bill of Materials – BOM), destacando não apenas a quantidade, mas também a função de cada item dentro do contexto didático do laboratório.

Componente	Quantidade	Justificativa didática
Nó: controle/acesso	1	Ponto central de operação e acesso dos alunos.
Nós: computação	10-30	Capacidade escalável por turma/projetos.
Servidor de storage	1	Dados comuns e diretórios de usuários.
Switch de rede	1-2	Conectividade e eventual segmentação de rede.

Quadro 2 - Exemplo de lista de componentes para laboratório

Fonte: Elaborado pelo autor (2026)

Enquanto o cenário de laboratório costuma privilegiar padronização e operação simples para atender múltiplos usuários com pouca variabilidade, ambientes corporativos frequentemente enfrentam um desafio adicional: crescer sem “quebrar” a arquitetura. A demanda por processamento pode variar por projetos, períodos do mês e ciclos de entrega, e o cluster precisa se adaptar sem exigir reconstruções frequentes. Por isso, além de dimensionar a capacidade, torna-se essencial organizar o cluster por camadas (serviço, computação e dados), de modo que cada parte possa evoluir de forma relativamente independente. O exemplo a seguir ilustra esse tipo de evolução planejada, típica de equipes de ciência de dados e MLOps.

- **Exemplo prático 2 (ambiente corporativo com crescimento):** considere uma equipe de ciência de dados que começa com um cluster pequeno para processamento de dados e inferência interna. No início, a prioridade é colocar o cluster em produção com pouca fricção, mas já prevendo crescimento. Um desenho recomendado é separar claramente a camada de serviço (acesso, APIs, notebooks) da camada de computação (nós de execução) e da camada de dados (storage e bancos). O resultado esperado é crescimento incremental: ao aumentar a demanda, adiciona-se nós de computação, expande-se o storage e aprimora-se a disponibilidade sem reescrever a arquitetura do zero.

Para reforçar o critério que guia esse desenho — crescer com equilíbrio, evitando gargalos —, a figura a seguir sintetiza a ideia de “caminho crítico” no desempenho do cluster.

Onde a capacidade é menor, surge o gargalo.
Planejar é alinhar capacidades ao fluxo de trabalho.

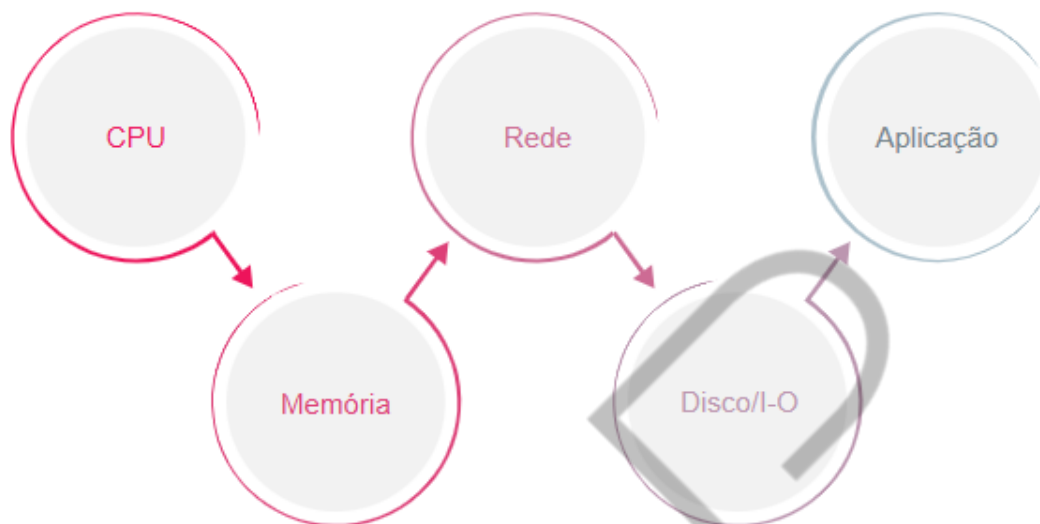


Figura 4 - Cadeia de gargalos: alinhando capacidades ao fluxo
Fonte: Elaborado pelo autor (2026)

Em termos práticos, ela ajuda a avaliar por que ampliar apenas CPU ou memória pode ter pouco efeito quando o limite real está em rede ou armazenamento, orientando investimentos e priorização de melhorias. Além disso, a figura reforça a ideia de que o desempenho do cluster é limitado pelo elo mais fraco do caminho. Deve-se notar que ajustes locais (por exemplo, mais CPU) podem não melhorar o sistema se a limitação estiver na rede ou no disco. Um planejamento saudável identifica o caminho crítico dos dados antes de investir em upgrades.

Boas práticas de fechamento da seção: ao final do planejamento, deve existir um documento simples de arquitetura contendo (i) objetivo do cluster, (ii) perfis de workload, (iii) papéis dos nós, (iv) desenho de rede e armazenamento, (v) política de crescimento (como expandir) e (vi) “lista de riscos” (o que pode falhar e como recuperar). Esse documento serve como referência comum entre quem usa e quem opera o cluster, reduzindo imprevisto e conflitos quando surgirem incidentes.

Resumo da seção (pontos-chave):

- O planejamento começa pelo workload e pelos piores casos de uso, não pela “lista de compras”;
- Separar papéis (controle, computação, storage) aumenta a previsibilidade e facilita a operação;

- Rede e storage são causas frequentes de gargalos em IA; dimensione pensando no caminho dos dados;
- Uma matriz de decisão ajuda a equilibrar custo, desempenho percebido e complexidade operacional;
- A arquitetura deve prever crescimento incremental, evitando “refazer tudo” ao escalar.

Checklist prático (antes de implementar):

- Defini o objetivo principal (desempenho, disponibilidade, serviços, dados) e o perfil de workload?
- Mapeei gargalos prováveis (CPU, memória, rede, I/O) e priorizei investimentos?
- Separei (ao menos logicamente) rede de gerenciamento e rede de dados quando necessário?
- Escolhi o armazenamento compatível com a criticidade dos dados e com a equipe disponível?
- Registre um desenho de referência e um plano de crescimento/backup?

2 GERENCIAMENTO DE CLUSTERS DE ALTA DISPONIBILIDADE

Alta Disponibilidade (High Availability – HA) não significa “nunca falhar”; significa “falhar sem interromper o serviço” ou, pelo menos, “falhar com interrupção mínima e controlada”. Em clusters, isso se aplica principalmente aos componentes que fazem o ambiente parecer “um sistema único”: acesso, coordenação, armazenamento e serviços críticos. O objetivo de HA é reduzir o tempo de indisponibilidade e tornar a recuperação previsível, mesmo quando falhas são inevitáveis (hardware, rede, atualização malsucedida ou erro humano).

2.1 Identificação de pontos críticos e failover

O primeiro passo de um desenho HA é identificar pontos únicos de falha (SPOF). Em clusters pequenos, o nó de controle, o servidor de armazenamento e o balanceador (quando existe) são candidatos óbvios a SPOF. Em muitos casos, não é possível eliminar todos os SPOFs sem aumentar custo e complexidade; portanto, a estratégia é priorizar o que afeta mais usuários e tem maior probabilidade de falhar. Essa priorização precisa ser explícita.

A figura a seguir apresenta o mecanismo mais didático para HA de acesso: um IP virtual que “aponta” para o nó ativo. Quando o nó ativo falha, o IP virtual migra para o nó de backup e os clientes continuam utilizando o mesmo destino. O leitor deve notar que esse padrão é simples, mas depende de boa detecção de falha e de regras claras para evitar alternâncias instáveis, conhecidas como flapping.

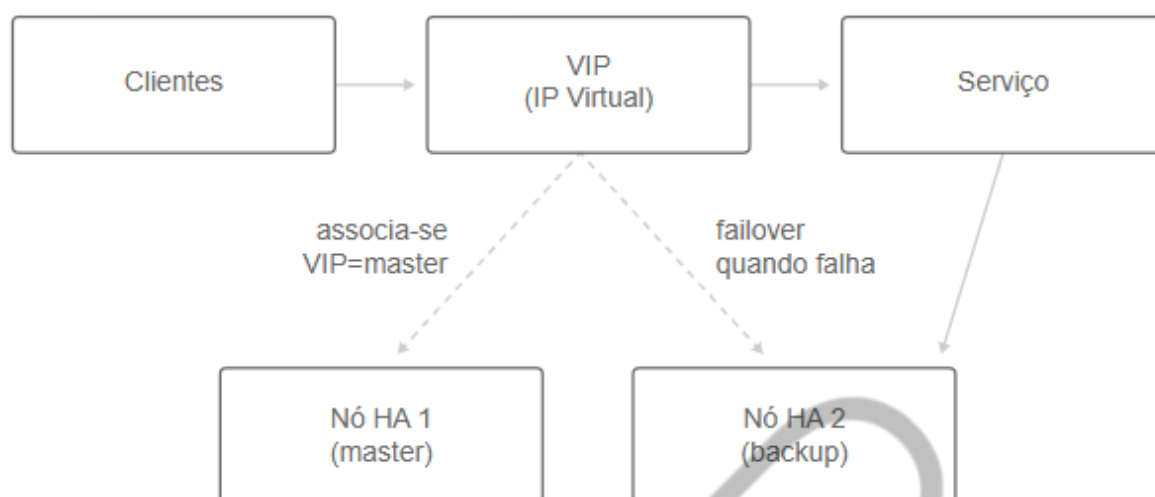


Figura 5 - Failover com IP Virtual (Virtual IP – VIP)

Fonte: Elaborado pelo autor (2026)

Em ambientes de Alta Disponibilidade, parte do desenho depende de um princípio simples: os nós precisam concordar sobre quem está “no comando” e qual estado do serviço é o válido. Quando essa coordenação se perde por causa de uma partição de rede (isto é, uma quebra de comunicação entre grupos de nós), surge um dos riscos mais críticos em clusters: o split-brain.

Nesse cenário, o cluster “se divide” em dois lados que passam a agir como se fossem o grupo correto, tentando assumir o serviço simultaneamente. O problema não é apenas haver dois “líderes” ao mesmo tempo; o dano real aparece quando cada lado realiza operações diferentes, gerando divergência de estado, por exemplo, gravações em disco, atualizações de banco, filas e registros que deixam de ser consistentes entre si.

2.2 Coordenação, quórum e modelos de redundância

A figura “Quórum e risco de split-brain em partição de rede” ilustra esse tipo de falha de forma intuitiva: um conjunto de nós permanece se enxergando como maioria e continua operando, enquanto outro conjunto, por não ter uma referência clara de quórum (isto é, uma regra objetiva de “maioria válida”), também pode decidir agir. O resultado típico é uma situação em que o serviço parece “de pé” em ambos os lados, mas com histórias diferentes do que aconteceu. Em termos práticos, isso pode se manifestar como dados duplicados, perda de transações, conflitos de versão e, em

casos extremos, necessidade de intervenção manual para escolher qual lado será mantido e qual precisará ser descartado ou reconstruído.

Por isso, a discussão de split-brain aparece sempre que se fala em quórum e failover: não basta trocar o serviço de um nó para outro; é essencial garantir que apenas um lado do cluster tenha autorização para operar quando há incerteza de comunicação. A figura a seguir serve como referência visual para entender por que mecanismos de quórum, verificação de saúde e políticas de isolamento são tão importantes no desenho de Alta Disponibilidade.

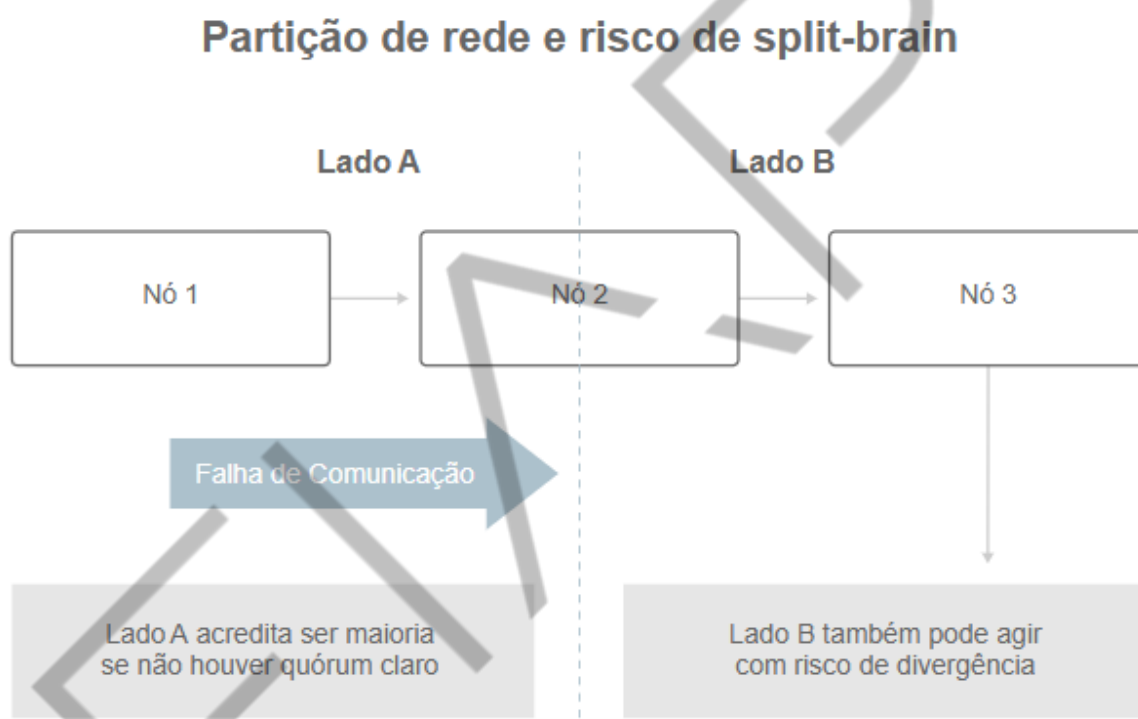


Figura 6 - Quórum e risco de split-brain em partição de rede
Fonte: Elaborado pelo autor (2026)

A discussão anterior sobre partição de rede e split-brain mostra que Alta Disponibilidade não se resume a “ter duas máquinas”. Redundância sem regras claras de coordenação pode gerar conflitos tão graves quanto a própria falha inicial. Por isso, uma estratégia sustentável de HA precisa equilibrar redundância técnica com simplicidade operacional. Não basta duplicar componentes; é necessário definir como eles se comunicam, quem assume em caso de falha e como evitar decisões simultâneas inconsistentes.

No caso de serviços de acesso e distribuição de requisições, um arranjo comum e eficaz é o padrão “dois nós + IP Virtual (VIP) + verificação de saúde (*health check*)”. Nesse modelo, há um ponto lógico estável para os clientes (o VIP), enquanto os nós

por trás dele podem alternar entre ativo e reserva conforme o estado do serviço. A verificação de saúde garante que apenas nós realmente operacionais recebam tráfego. Esse desenho reduz interrupções e mantém previsibilidade, desde que exista monitoramento contínuo e procedimentos de manutenção bem definidos.

Para a camada de dados, entretanto, a abordagem exige ainda mais cuidado. Dependendo do tipo de aplicação, pode-se optar por replicação síncrona ou assíncrona, armazenamento distribuído ou estratégias robustas de backup com restauração rápida. A escolha deve considerar criticidade, volume de dados e tolerância à perda. A regra geral é clara: quanto mais crítico o dado, mais a consistência e a recuperação devem estar incorporadas ao projeto arquitetural, e não tratadas como etapa posterior ou contingencial.

A figura “Remoção de SPOF com redundância N+1” ilustra essa evolução conceitual: sair de um cenário com ponto único de falha (SPOF) para uma arquitetura com redundância planejada (N+1). No primeiro caso, um único componente intermediário concentra risco; no segundo, a presença de múltiplos elementos elimina dependência exclusiva e permite continuidade mesmo diante de falhas individuais.

A imagem sintetiza visualmente o princípio discutido: HA não é apenas duplicar recursos, mas reorganizá-los para que o sistema continue operando de forma controlada quando algo inevitavelmente falhar.

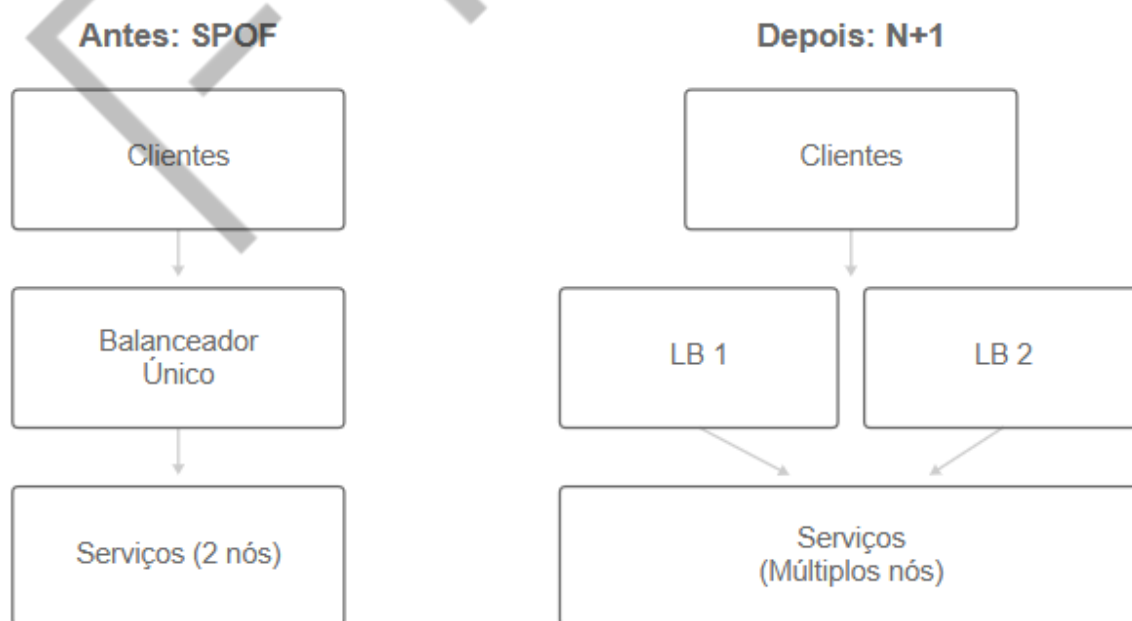


Figura 7 - Remoção de SPOF com redundância N+1
Fonte: Elaborado pelo autor (2026)

A figura anterior evidencia que a simples eliminação de um ponto único de falha não encerra a discussão sobre Alta Disponibilidade. A redundância precisa ser organizada segundo um padrão coerente de funcionamento: quem assume, em que condições, com quais garantias de consistência e com qual impacto operacional. Em outras palavras, a arquitetura deixa de ser apenas estrutural (quantos nós existem) e passa a ser também comportamental (como esses nós reagem a falhas).

É nesse momento que se torna necessário diferenciar os modelos de organização da redundância. Nem todo serviço exige o mesmo arranjo, e nem toda infraestrutura comporta o mesmo nível de complexidade. Serviços com estado, aplicações altamente escaláveis, ambientes com crescimento incremental ou estruturas distribuídas fisicamente demandam estratégias distintas. A escolha do padrão adequado depende do tipo de carga, da criticidade dos dados e da maturidade operacional da equipe responsável.

Para sistematizar essas alternativas e tornar a decisão menos intuitiva e mais estruturada, o quadro “Padrões de HA e quando usar (visão qualitativa)” apresenta os principais padrões de Alta Disponibilidade e indica, de forma qualitativa, em que contextos cada um tende a ser mais apropriado, bem como os cuidados que exigem.

Padrão	Quando faz sentido	Atenções
Ativo-passivo	Serviços com estado ou custo alto por duplicação	Tempo de failover e testes de retorno
Ativo-ativo	Serviços stateless e tráfego alto	Consistência de sessão e balanceamento
N+1	Ambientes que crescem por adição de nós	Monitorar capacidade residual após falha
Multi-zona	Exigência de resiliência a falhas de sala/energia	Latência e custo de replicação

Quadro 3 - Padrões de HA e quando usar (visão qualitativa)

Fonte: Elaborado pelo autor (2026)

Após organizar os padrões de Alta Disponibilidade no quadro anterior, surge uma etapa fundamental para consolidar o entendimento: traduzir esses modelos em situações concretas. Conceitos como ativo-passivo, N+1 ou multi-zona ganham clareza quando observados em cenários reais de aplicação. A análise qualitativa permite compreender não apenas como estruturar a redundância, mas porque determinadas escolhas são feitas em função do tipo de serviço, do risco envolvido e da experiência do usuário final.

A seguir, dois contextos distintos ilustram como as decisões arquiteturais discutidas anteriormente se materializam em ambientes práticos.

- **Exemplo prático 1 – Serviço Web com Continuidade Automática:** considere uma aplicação web utilizada para disponibilizar notebooks, dashboards ou APIs internas. O objetivo não é eliminar toda falha possível, mas garantir que a indisponibilidade percebida seja mínima. Nesse cenário, dois servidores executam a aplicação simultaneamente ou em regime ativo-reserva, enquanto um ponto de entrada estável direciona as requisições. Caso um servidor deixe de responder, o tráfego é automaticamente redirecionado para o outro. O aspecto central aqui não é a quantidade de máquinas, mas o comportamento do sistema diante da falha. O usuário continua acessando o mesmo endereço; a troca ocorre internamente. Esse desenho é indicado para aplicações que podem ser replicadas com facilidade e cujo estado não depende exclusivamente de um único processo em memória.

A figura “Arquitetura web com IP Virtual e nó de reserva (modelo de Alta Disponibilidade)” ilustra diagramaticamente um serviço web em Alta Disponibilidade no qual os clientes acessam um ponto único (IP Virtual), enquanto dois servidores garantem continuidade automática. Caso o nó principal falhe, o nó de reserva assume o atendimento sem alteração do endereço de acesso.

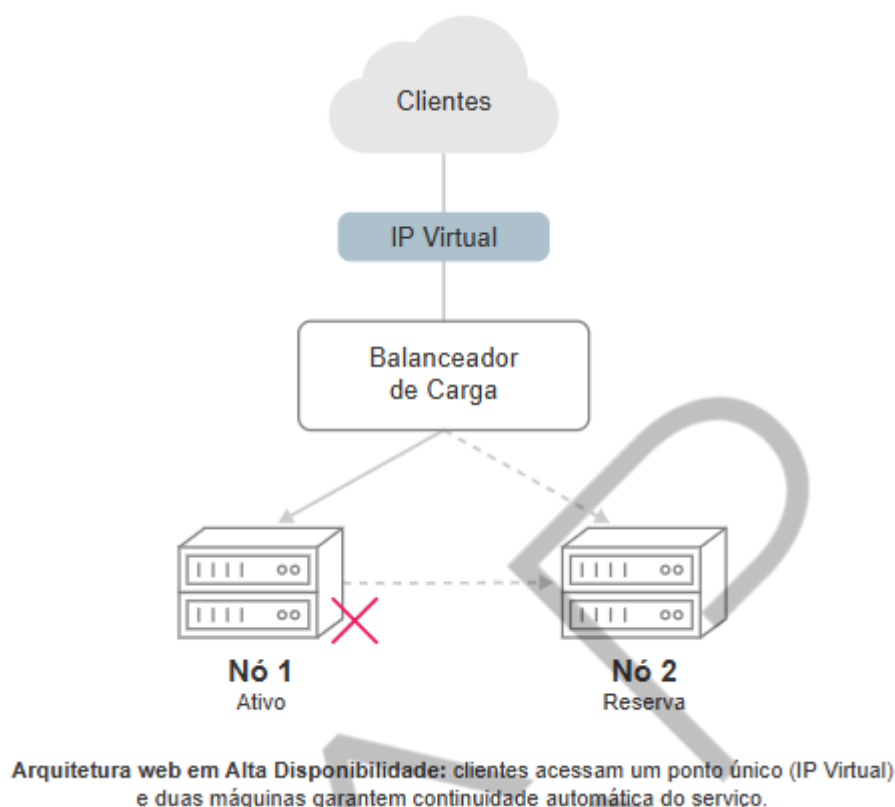


Figura 8 - Arquitetura web com IP Virtual e nó de reserva (modelo de Alta Disponibilidade)
Fonte: Elaborado pelo autor (2026)

- **Exemplo prático 2 – Serviço Crítico com Dependência de Dados:** agora considere um repositório de dados compartilhado por múltiplos projetos de Inteligência Artificial. Aqui, manter a aplicação “no ar” não é suficiente. É necessário garantir que os dados permaneçam consistentes, mesmo em situações adversas. Nesse contexto, a arquitetura costuma separar claramente dois planos: a) Plano de continuidade do serviço: assegurar que exista sempre um ponto de acesso disponível; b) Plano de integridade dos dados: garantir replicação controlada, mecanismos de consenso e procedimentos de recuperação. Essa separação evita que uma troca automática de nó provoque divergência de informações. A preocupação deixa de ser apenas disponibilidade e passa a incluir integridade e coerência do estado.

A figura “Arquitetura de Alta Disponibilidade para repositório de dados crítico com replicação” ilustra a separação entre dois níveis distintos da arquitetura. Na parte superior, a aplicação representa o ponto estável de acesso para os clientes, que

continuam enviando requisições independentemente de qual nó esteja ativo na camada de dados.

Na parte inferior, observa-se o conjunto de servidores responsáveis pelo armazenamento, organizados de forma que apenas um nó principal aceite escritos, enquanto os demais mantêm réplicas atualizadas. O elemento central do diagrama é o quórum, que atua como mecanismo de decisão coletiva. Ele impede que múltiplos nós assumam simultaneamente o papel principal em caso de partição de rede, mitigando o risco de divergência.

Dessa forma, a continuidade do serviço não compromete a integridade das informações. A arquitetura demonstra que, em sistemas críticos, disponibilidade e consistência precisam ser tratadas de forma integrada, combinando coordenação, replicação e procedimentos de recuperação bem definidos.

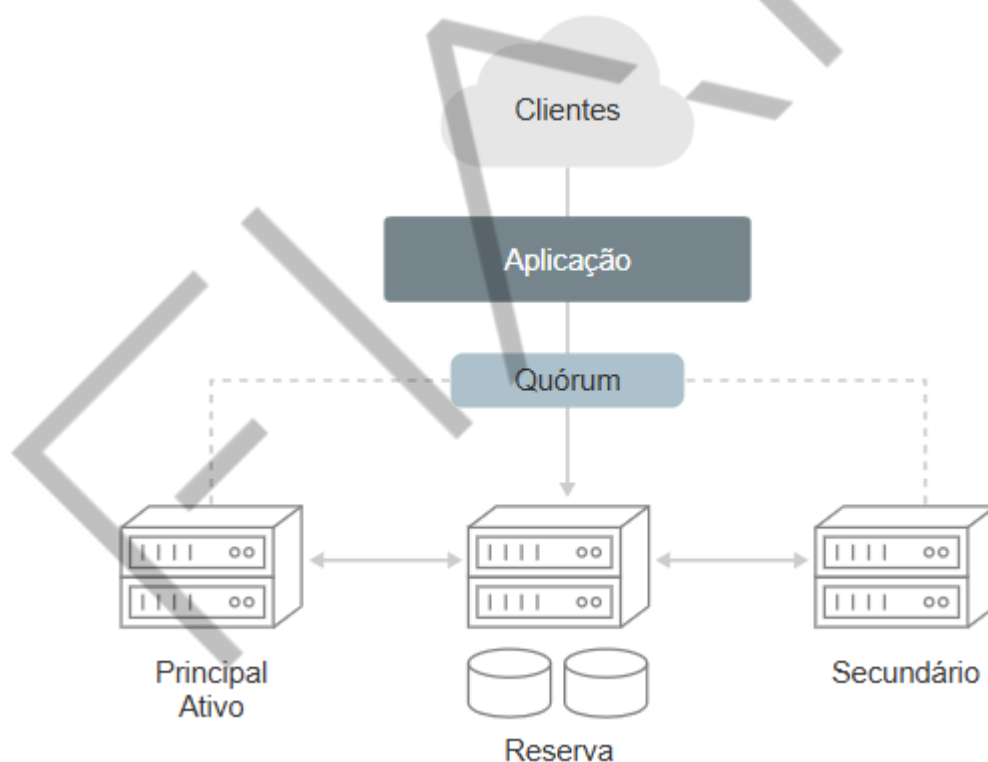


Figura 9 - Arquitetura de Alta Disponibilidade para repositório de dados crítico com replicação controlada

Fonte: Elaborado pelo autor (2026)

Resumo da seção (pontos-chave):

- Alta disponibilidade reduz o impacto de falhas, mas não elimina falhas;
- Identificar e priorizar SPOFs é o primeiro passo antes de escolher as ferramentas;

- Failover depende de detecção (*health checks*/heartbeats), decisão (quórum) e ação (troca);
- Split-brain é risco crítico em serviços com estado; quórum e contenção são essenciais;
- A HA sustentável combina redundância com operação simples e testes periódicos.

Checklist prático (operar HA com segurança):

- Liste componentes críticos e classifiquei impacto de falha (quem é SPOF)?
- Defini mecanismo de detecção e tempo aceitável de recuperação para cada serviço?
- Existe regra clara de quórum e proteção contra split-brain para serviços com estado?
- Testei failover e retorno (failback) e registrei procedimentos?
- Monitorei e alertei eventos (falha, recuperação, flapping) com responsáveis definidos?

3 BALANCEAMENTO DE CARGA E EFICIÊNCIA

Balanceamento de carga é o mecanismo que distribui requisições ou tarefas entre vários recursos disponíveis, evitando que apenas um deles fique sobrecarregado enquanto outros permanecem ociosos. Em termos simples, é como organizar uma fila para que várias pessoas atendam clientes ao mesmo tempo, em vez de apenas uma. O objetivo é melhorar a experiência do usuário, manter estabilidade e reduzir risco de interrupções.

3.1 Estratégias de distribuição de carga

Em clusters que oferecem serviços, como APIs ou ambientes de notebooks, o balanceamento decide para qual servidor cada requisição será enviada. Já em clusters de processamento, o mesmo princípio aparece no agendamento de tarefas e na organização do acesso ao armazenamento e à rede. Em qualquer cenário, a ideia central permanece: distribuir a carga de forma equilibrada.



Figura 10 - L4 versus L7: onde o balanceamento decide
Fonte: Elaborado pelo autor (2026)

Na parte esquerda da figura “L4 *versus* L7: onde o balanceamento decide”, temos o Balanceamento L4 (Transporte). Nesse modelo, o balanceador toma decisões olhando apenas informações básicas da conexão, como endereço IP e número de porta. Ele não “entende” o conteúdo da requisição; apenas distribui conexões entre servidores disponíveis. Por isso, tende a ser mais rápido e exigir menos processamento adicional. É indicado quando o foco principal é desempenho e simplicidade.

Já na parte direita da figura “L4 *versus* L7: onde o balanceamento decide”, vemos o Balanceamento L7 (Aplicação). Nesse caso, o balanceador analisa elementos da própria requisição, como a URL acessada, cabeçalhos ou até partes do conteúdo. Isso permite decisões mais inteligentes — por exemplo, enviar requisições de “/api” para um grupo específico de servidores e requisições de “/relatorios” para outro. Contudo, essa análise exige mais processamento, aumentando a complexidade.

Observações que devem ser notadas na figura “L4 *versus* L7: onde o balanceamento decide”:

- No L4, a decisão ocorre na camada de transporte (conexão).
- No L7, a decisão ocorre na camada de aplicação (conteúdo da requisição).
- Quanto mais “alto” na pilha a decisão acontece, maior o nível de entendimento da requisição, e maior o custo computacional.

Sendo assim, a escolha entre L4 e L7 não é sobre qual é “melhor”, mas sobre qual atende melhor ao contexto: desempenho simples e rápido ou controle mais refinado sobre o tráfego.

Outra distinção importante ao discutir crescimento de um cluster é entender como aumentar sua capacidade. Existem duas estratégias principais: escalabilidade vertical e escalabilidade horizontal.

A escalabilidade vertical consiste em fortalecer um único servidor, adicionando mais processador (CPU), mais memória ou armazenamento mais rápido. É uma solução simples e direta, mas tem dois limites claros: existe um teto físico de expansão e todo o serviço continua concentrado em um único ponto. Se esse servidor falhar, todo o sistema é afetado.

Já a escalabilidade horizontal amplia a capacidade adicionando novos servidores ao conjunto. Em vez de um equipamento muito potente, temos vários trabalhando juntos. Essa abordagem reduz a concentração de risco e permite o crescimento gradual. No entanto, para funcionar corretamente, o sistema precisa saber como distribuir a carga entre esses nós adicionais. É nesse ponto que entram os balanceadores e as práticas de projeto, como manter serviços sem dependência de estado local (stateless), utilizar cache compartilhado, replicação de dados e particionamento.

A figura a seguir ajuda a visualizar essa decisão de forma estruturada. Ela mostra que a escolha do algoritmo não é aleatória: depende do comportamento do tráfego.

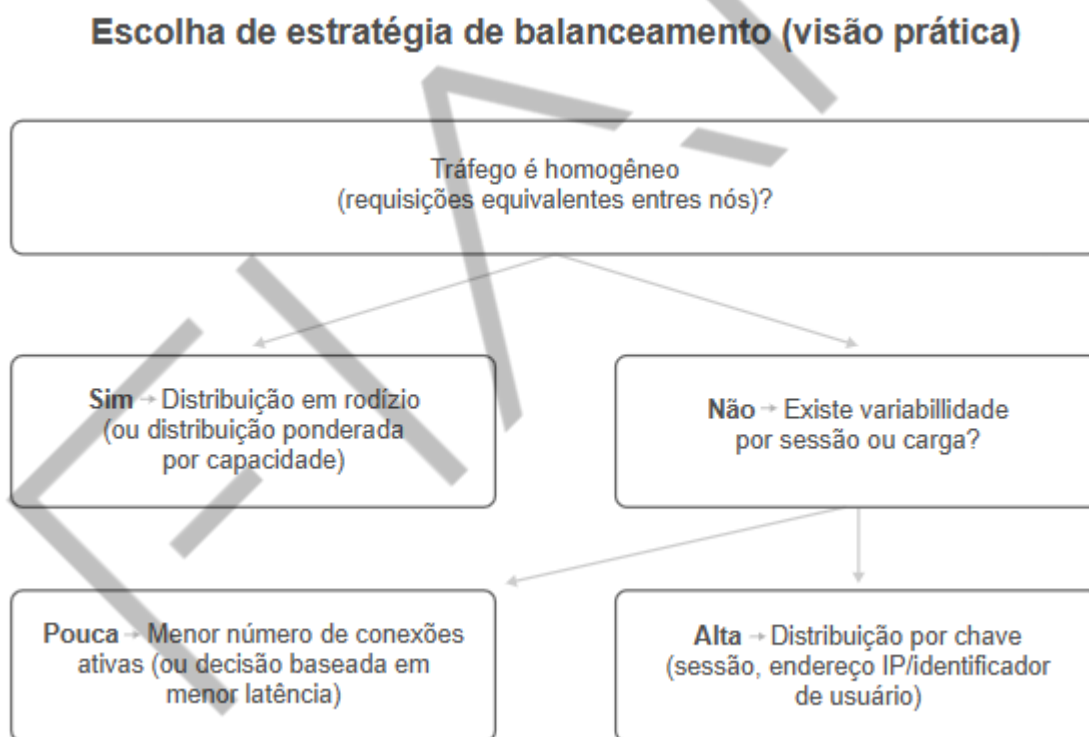


Figura 11 - Estratégia de balanceamento: decisão por características do tráfego
Fonte: Elaborado pelo autor (2026)

No topo da figura, a pergunta é simples: o tráfego é homogêneo? Ou seja, todas as requisições exigem esforço semelhante dos servidores?

Se a resposta for “sim”, um algoritmo como round-robin (rodízio simples) pode ser suficiente. Ele distribui as requisições alternadamente entre os nós, garantindo equilíbrio básico.

Se o tráfego não for homogêneo, por exemplo, algumas requisições exigem mais processamento que outras, o diagrama orienta a avaliar a variabilidade por sessão ou carga:

- a) Quando a diferença entre requisições é pequena, mas existe alguma variação, o algoritmo least-connections (menor número de conexões ativas) pode ser mais adequado. Ele direciona novas requisições para o servidor que está menos ocupado no momento.
- b) Quando a variabilidade é alta e há dependência de sessão, o hashing por chave (como por sessão, IP ou identificador de usuário) torna-se necessário. Nesse modelo, requisições relacionadas ao mesmo usuário tendem a ir sempre para o mesmo servidor, preservando consistência.

Além disso, o estudante deve observar dois fatores práticos destacados no texto:

1. **Capacidade desigual entre nós:** se os servidores não possuem o mesmo desempenho, é necessário aplicar pesos (por exemplo, enviar mais requisições para máquinas mais potentes).
2. **Dependência de sessão:** se o serviço mantém informações temporárias na memória local, a estratégia precisa garantir afinidade ou então externalizar esse estado para um armazenamento compartilhado.

Dessa forma, foi possível verificar que escolher um algoritmo de balanceamento não é questão de preferência técnica, mas de análise do comportamento do sistema. A decisão correta depende da forma como o tráfego se comporta e de como o serviço foi projetado para crescer.

3.2 Eficiência operacional e gestão de capacidade

Depois de entender por que a distribuição de tráfego é necessária e como escolher uma estratégia em função do perfil das requisições (homogêneas ou variáveis, com ou sem afinidade), falta dar um passo prático: transformar essa lógica em critérios de escolha de algoritmo. Em um ambiente real, o desafio não é decorar nomes, e sim reconhecer o comportamento do sistema e selecionar a abordagem que tende a produzir estabilidade no dia a dia.

Uma forma simples de fazer isso é pensar no balanceamento como uma “regra de encaminhamento”. Essa regra responde, essencialmente, a duas perguntas: o que significa “justo” para o seu serviço (mesmo número de requisições, mesmo tempo de conexão, mesmo custo de processamento)? E o que precisa ser preservado (por exemplo, afinidade de sessão ou distribuição pura)? Como esses objetivos podem entrar em conflito, a escolha correta costuma ser aquela que minimiza surpresas: picos de latência, servidores saturados ou comportamento inconsistente em horários de maior demanda.

Nesse ponto, o quadro “Algoritmos comuns de balanceamento (visão aplicada)” funciona como uma referência rápida: ela reúne os algoritmos mais comuns e associa cada um a uma ideia central, ao tipo de cenário em que costuma funcionar bem e ao principal cuidado a ser observado. O objetivo não é apresentar “um vencedor”, mas mostrar que cada alternativa resolve um problema específico — e pode falhar quando aplicada fora do contexto para o qual foi pensada.

Por exemplo, um rodízio simples pode parecer equilibrado no início, mas ignorar diferenças reais de custo entre requisições; já escolher consistência por chave pode resolver sessão, mas criar concentração de carga se muitas requisições caírem no mesmo grupo. Ao ler o quadro, deve-se sempre conectar “quando usar” com “limitação típica”, pois é justamente essa combinação que orienta decisões mais maduras.

Algoritmo	Ideia	Quando usar	Limitação típica
Distribuição em rodízio	Rodízio simples	Serviços homogêneos, stateless.	Ignora diferenças de carga real.
Menor número de conexões ativas	Menos conexões ativas	Conexões com durações variadas.	Métrica pode não refletir custo real.
Distribuição por chave	Consistência por chave	Afinidade de sessão/estado.	Pode criar hotspots se a chave for enviesada.
Pesos	Capacidade diferente	Nós heterogêneos.	Requer calibrar e revisar pesos.

Quadro 4 - Algoritmos comuns de balanceamento (visão aplicada)
Fonte: Elaborado pelo autor (2026)

A eficiência é influenciada pelo balanceamento, mas depende também de dimensionamento adequado, organização de dados e políticas operacionais. Não se trata apenas de distribuir requisições ou tarefas entre vários nós, mas de garantir que essa distribuição resulte em uso estável e previsível dos recursos disponíveis.

Em um cluster, eficiência se manifesta quando CPU, memória, rede e armazenamento operam em níveis saudáveis, sem ociosidade excessiva e sem aproximação constante do limite máximo. O objetivo não é “extrair 100%” da infraestrutura o tempo todo, mas manter um ponto de operação que permita absorver variações de carga sem perda significativa de desempenho.

Um erro recorrente em ambientes iniciantes é confundir eficiência com utilização máxima. À primeira vista, pode parecer desejável manter todos os nós sempre no limite da capacidade. No entanto, sistemas operando muito próximos da saturação tornam-se sensíveis a pequenas variações de demanda. Um aumento modesto de requisições pode desencadear crescimento desproporcional no tempo de resposta, formação de filas internas e instabilidade perceptível pelos usuários. Assim, eficiência sustentável envolve margem operacional, isto é, capacidade reservada para acomodar picos inesperados. A figura a seguir ilustra exatamente esse comportamento.

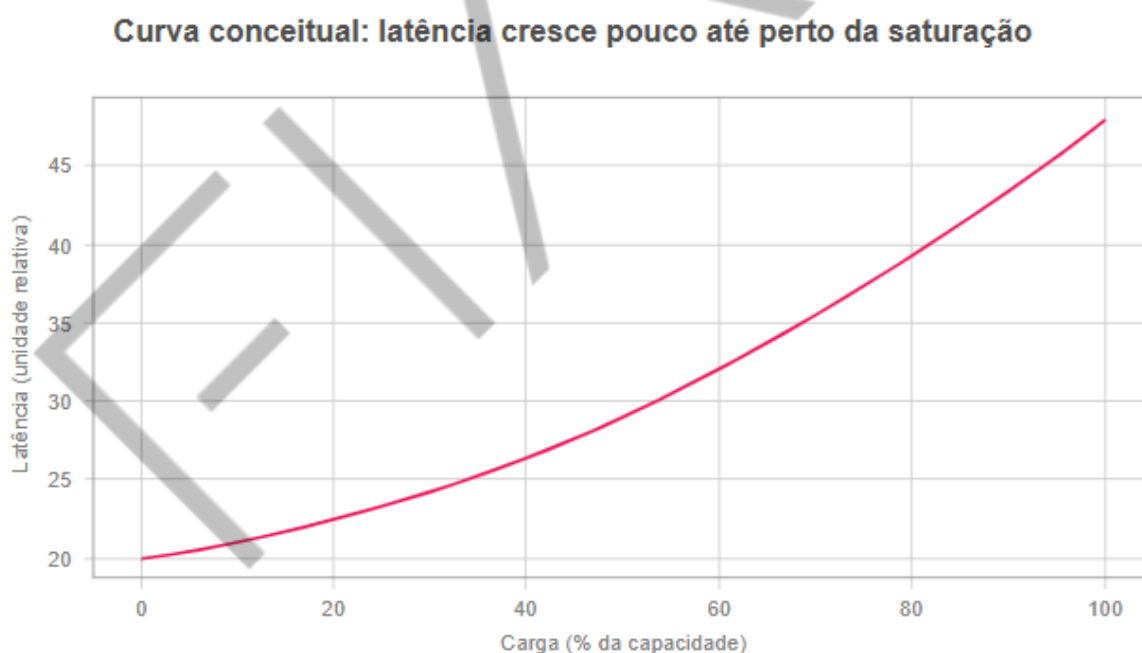


Figura 12 - Curva conceitual de latência *versus* carga
Fonte: Elaborado pelo autor (2026)

No eixo horizontal, observa-se a carga do sistema como percentual da capacidade total. No eixo vertical, a latência relativa (tempo de resposta) cresce de forma moderada até certo ponto, mas passa a aumentar rapidamente quando a utilização se aproxima do limite máximo. A curva conceitual mostra que, enquanto o sistema opera em faixa intermediária, pequenos acréscimos de carga produzem

impacto limitado. Entretanto, quando se aproxima da saturação, a inclinação da curva se acentua: variações mínimas geram aumentos expressivos na latência. Esse fenômeno explica por que ambientes aparentemente “bem aproveitados” podem apresentar degradação súbita sob demanda elevada.

Portanto, eficiência não significa utilizar o cluster no limite permanente, mas manter operação estável com margem para absorver picos e falhas parciais. Essa visão conecta balanceamento, escalabilidade e gestão de capacidade em uma mesma lógica operacional.

O cache pode ser compreendido como um mecanismo complementar ao balanceamento e ao dimensionamento. Se o balanceamento organiza o fluxo de requisições e a eficiência define o ponto saudável de operação, o cache atua reduzindo a quantidade de trabalho que precisa, de fato, ser executada. A lógica é direta: se uma resposta já foi produzida e continua válida, não há motivo para recalculá-la. Essa reutilização diminui o processamento, reduz acessos a disco e melhora o tempo de resposta percebido pelo usuário.

Em serviços web, o cache pode armazenar conteúdos estáticos (imagens, folhas de estilo, scripts) ou até respostas completas de determinadas requisições. Em ambientes de dados e Inteligência Artificial, pode guardar resultados intermediários de processamento, conjuntos de dados já transformados ou consultas frequentes. Em ambos os casos, a função é a mesma: interceptar parte da demanda antes que ela atinja os componentes mais sensíveis da infraestrutura. A figura a seguir sintetiza esse papel de forma clara.

Cache reduz pressão no backend e melhora latência percebida.

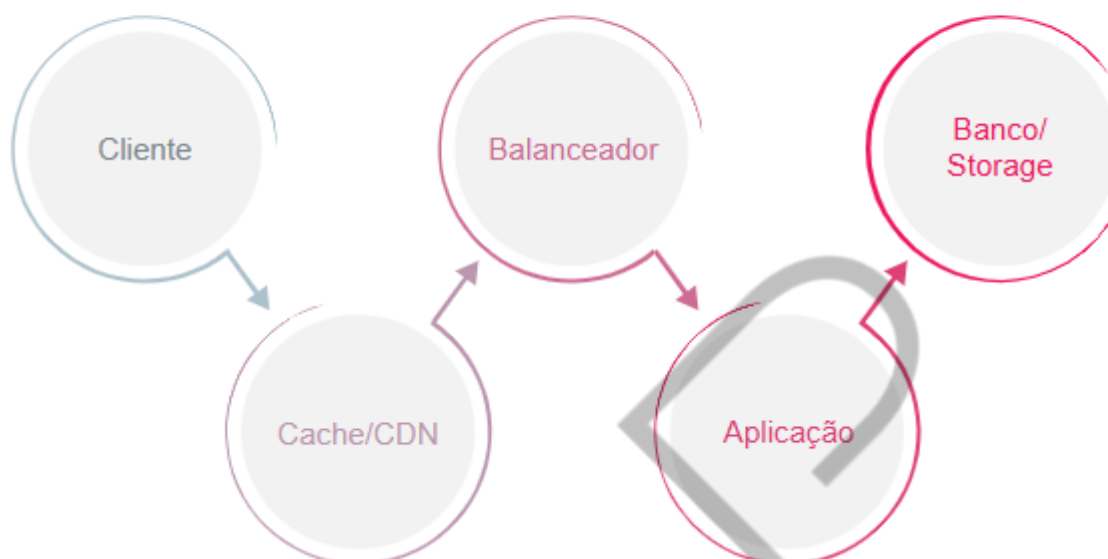


Figura 13 - Pipeline de cache: reduzindo pressão no backend
Fonte: Elaborado pelo autor (2026)

Nesta figura, observa-se que o fluxo de requisição parte do cliente e encontra, logo no início, a camada de **Cache/CDN** (Content Delivery Network). Apenas as requisições que não podem ser atendidas pelo cache seguem para o balanceador, depois para a aplicação e, por fim, para o banco de dados ou sistema de armazenamento. O ponto central da imagem é mostrar que o cache não substitui a aplicação, mas funciona como um filtro que absorve parte significativa do tráfego, reduzindo a carga nas camadas posteriores.

Essa redução de pressão é especialmente relevante para banco e armazenamento, que costumam ser os elementos mais custosos e mais difíceis de escalar. Enquanto adicionar servidores de aplicação pode ser relativamente simples em uma arquitetura horizontal, ampliar a capacidade de um banco de dados exige planejamento adicional, replicação e cuidados com consistência. Ao diminuir o número de acessos repetidos, o cache prolonga a faixa de operação estável discutida anteriormente na curva de latência.

Portanto, o uso de cache não é apenas uma otimização pontual, mas uma estratégia estrutural de eficiência. Ele amplia a capacidade efetiva do cluster sem necessariamente aumentar o número de nós. Quando combinado com balanceamento adequado e monitoramento de carga, o cache contribui para manter

o sistema distante da zona de saturação, preservando desempenho e estabilidade mesmo sob crescimento gradual da demanda.

Até este ponto da discussão, foram apresentados os fundamentos de balanceamento, eficiência e estratégias de crescimento. No entanto, esses conceitos ganham sentido real quando aplicados a cenários concretos. Em ambientes modernos, especialmente em startups e laboratórios acadêmicos, a diferença entre um sistema estável e um sistema imprevisível costuma estar na forma como a carga é distribuída e como os recursos são organizados.

Os exemplos a seguir mostram duas situações comuns: uma equipe pequena com serviços de inferência e um laboratório com uso concentrado em determinados horários. Ambos ilustram que desempenho não depende apenas de potência computacional, mas da organização do fluxo de requisições e da gestão de gargalos invisíveis.

- **Exemplo prático 1 – Startup com APIs internas:** imagine uma startup que mantém três serviços de inferência para modelos de IA. Inicialmente, cada serviço roda em um único servidor. Conforme o número de requisições cresce, um deles passa a concentrar mais chamadas e se torna um gargalo. A latência aumenta e a experiência dos usuários se degrada. A solução não é simplesmente “aumentar a máquina”, mas reorganizar a arquitetura. Introduce-se um ponto de entrada que distribui as requisições entre múltiplas instâncias do mesmo serviço. Quando a demanda cresce, novas réplicas podem ser adicionadas. A capacidade deixa de ser vista como potência de um servidor isolado e passa a ser entendida como quantidade de instâncias disponíveis. Essa abordagem espelha o que é amplamente adotado em ambientes de nuvem, como o uso de Application Load Balancer + Auto Scaling Groups na AWS.

A figura “Arquitetura de escalabilidade horizontal com balanceador e múltiplas instâncias (modelo inspirado em AWS)” ilustra exatamente essa situação, ou seja, um ambiente em que um ponto único de entrada (balanceador de aplicação) distribui requisições entre múltiplas instâncias do mesmo serviço de inferência. A capacidade do sistema cresce por adição de réplicas, permitindo absorver aumento de demanda

sem concentrar carga em um único servidor. O banco ou armazenamento permanece como camada compartilhada, acessada de forma controlada pelas instâncias.

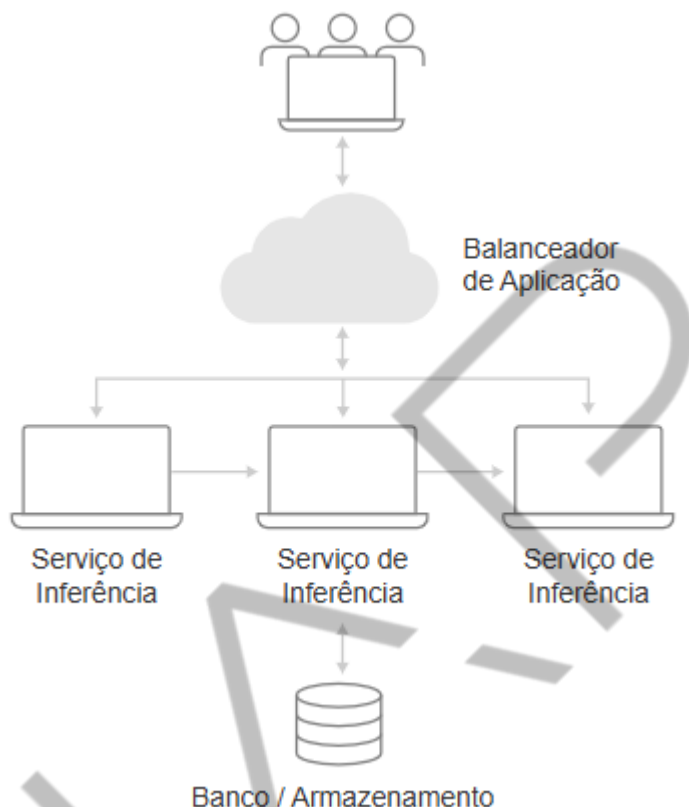


Figura 14 - Arquitetura de escalabilidade horizontal com balanceador e múltiplas instâncias (modelo inspirado em AWS)

Fonte: Elaborado pelo autor (2026)

- **Exemplo prático 2 – Laboratório com múltiplas turmas:** em um laboratório acadêmico, o cenário é diferente. O cluster pode aparentar estar subutilizado durante a maior parte do dia, mas nos horários de entrega de atividades ocorre um pico de uso. Muitos alunos executam tarefas simultaneamente, acessando o mesmo conjunto de dados. O problema não está necessariamente na CPU, mas no congestionamento de armazenamento e rede. Nesse caso, eficiência passa por reduzir “pontos quentes” (hotspots). Estratégias simples incluem distribuir os conjuntos de dados em múltiplos locais, utilizar cache local nos nós de computação e aplicar políticas de fila para organizar execuções concorrentes. O ganho não está em aumentar o hardware, mas em reorganizar o fluxo de acesso.

A figura “Organização de acesso a dados em laboratório acadêmico com múltiplas turmas” apresenta de forma esquemática exatamente essa situação, isto é,

um cenário em que múltiplos usuários executam tarefas simultaneamente, utilizando um sistema de filas para organizar requisições e nós de computação com cache local para reduzir pressão sobre o armazenamento central. A estrutura evidencia que o desempenho não depende apenas de CPU, mas também da organização do acesso a dados e da mitigação de congestionamentos em armazenamento e rede.

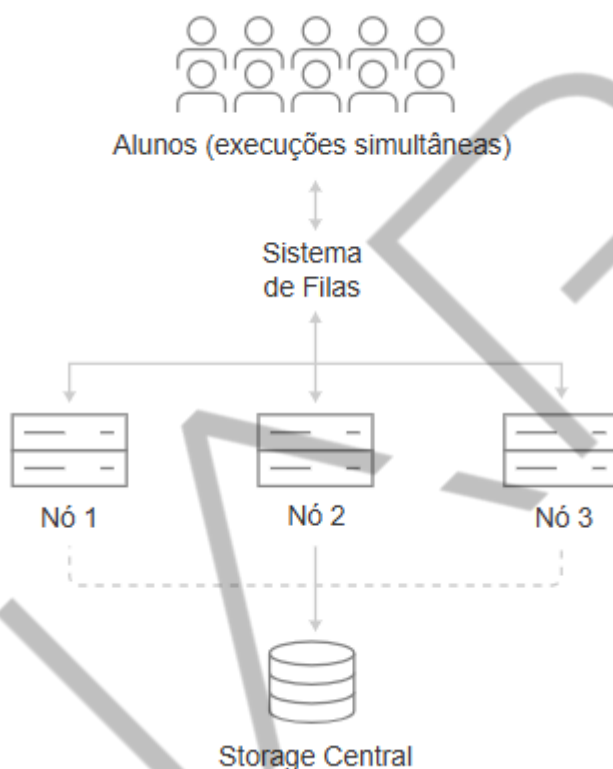


Figura 15 - Organização de acesso a dados em laboratório acadêmico com múltiplas turmas
Fonte: Elaborado pelo autor (2026)

Após os dois exemplos, a startup que cresce por adição de réplicas e o laboratório que organiza melhor o acesso aos dados, fica claro que eficiência não depende de uma única decisão estrutural. Ela resulta de um conjunto de ajustes que atuam em diferentes pontos da arquitetura. Em alguns casos, a melhoria vem da forma como os serviços são organizados; em outros, da forma como o uso é regulado ou da maneira como o consumo de recursos é planejado ao longo do tempo.

Para sistematizar essas possibilidades, o quadro a seguir reúne um conjunto de alavancas práticas que podem ser aplicadas conforme o contexto. Elas não substituem balanceamento ou escalabilidade, mas complementam essas estratégias, ajudando a manter o sistema em uma zona estável de operação. O ponto central é

observar que cada alavanca melhora um aspecto específico do ambiente, e traz consigo um cuidado associado.

No exemplo da startup, por exemplo, o uso de autoscaling permite ajustar o número de instâncias conforme a demanda, transformando crescimento em processo controlado. Já no laboratório, limites e cotas ajudam a distribuir recursos de forma mais justa durante picos de uso. Em ambos os cenários, cache aparece como mecanismo transversal, reduzindo acessos repetidos e protegendo camadas mais sensíveis da infraestrutura.

Alavanca	O que melhora	Cuidado
Consolidar serviços	Menos ociosidade	Evitar acoplamento excessivo
Cache (conteúdo/dados)	Menos I/O e latência	Consistência e invalidação de cache
Limites e cotas	Justiça e previsibilidade	Exige comunicação com usuários
Autoscaling	Crescimento sob demanda	Custo e complexidade de operação
Desligamento programado	Energia em períodos ociosos	Impacto em serviços

Quadro 5 - Alavancas práticas de eficiência em clusters (qualitativas)

Fonte: Elaborado pelo autor (2026)

Resumo da seção (pontos-chave):

- Balancear carga evita hotspots e aumenta previsibilidade em serviços e em computação;
- L4 tende a ser mais simples e rápido; L7 tende a ser mais flexível e expressivo;
- Escalabilidade horizontal é a estratégia típica de crescimento em clusters;
- Algoritmos (round-robin, least-connections, hashing) dependem de tráfego e estado;
- Eficiência não é uso máximo; é operar com margem para picos e evitar saturação.

Checklist prático (antes de considerar o ambiente “eficiente”):

- Entendo se meu caso é mais bem atendido por L4 ou L7 (ou combinação)?
- Escolhi o algoritmo de balanceamento compatível com estado/sessão e heterogeneidade?

- Tenho métricas básicas de latência, throughput e utilização para comparar mudanças?
- Identifiquei hotspots de I/O/rede e apliquei organização/caching quando cabível?
- Mantive margem de capacidade para picos e eventos de falha (planejamento N+1)?

EMBA

4 GERENCIAMENTO DE DESEMPENHO E SEGURANÇA EM CLUSTERS

Depois que um cluster entra em operação, a atenção deixa de estar apenas na arquitetura e passa a se concentrar em duas perguntas que guiam o cotidiano: o ambiente mantém desempenho previsível e permanece protegido contra riscos comuns. Essas dimensões não evoluem de forma independente. Um ajuste voltado a reduzir latência pode introduzir exposição (por exemplo, portas abertas sem necessidade ou permissões amplas), enquanto controles de segurança podem adicionar etapas e custos computacionais (como auditoria, verificação de integridade e criptografia). A gestão madura busca um equilíbrio: preservar o desempenho sem comprometer os controles, e fortalecer os controles sem tornar o ambiente impraticável.

Esse equilíbrio é sustentado por três elementos: **observabilidade, processos operacionais e revisões periódicas**. Observabilidade fornece evidência do que está acontecendo; processos garantem que respostas a incidentes e mudanças sejam consistentes; revisões evitam que configurações antigas permaneçam “por inércia” mesmo quando o contexto muda. Em clusters, onde múltiplos nós e serviços interagem, a ausência desses pilares costuma levar a dois problemas: decisões tomadas no escuro e correções reativas que resolvem um sintoma, mas criam novos gargalos ou vulnerabilidades.

4.1 Observabilidade e diagnóstico de desempenho

Observabilidade pode ser definida como a capacidade de inferir o estado interno do sistema a partir de sinais externos. Na prática, ela se apoia em três pilares: **registros** (logs), **métricas** e, quando aplicável, **rastreamento distribuído** (traces). Logs descrevem eventos e ajudam a reconstruir o que aconteceu; métricas mostram tendências e variações ao longo do tempo; traces conectam etapas de uma requisição ponta a ponta, útil em arquiteturas mais complexas. Em ambientes iniciais, logs e métricas já geram grande impacto, pois permitem identificar rapidamente se o problema é saturação de CPU, contenção de disco, fila de rede, falta de memória ou falhas intermitentes entre nós.

O objetivo principal é reduzir o tempo entre “algo ficou ruim” e “entendemos por quê”. Isso significa detectar degradações antes que virem incidentes graves e, quando uma falha ocorrer, explicar sua causa com rapidez suficiente para agir de forma precisa. Em um cluster, essa capacidade é essencial tanto para desempenho quanto para segurança: a mesma instrumentação que revela um gargalo pode também sinalizar comportamentos anômalos, como acessos fora do padrão, aumento repentino de tentativas de autenticação ou tráfego inesperado entre segmentos de rede. Assim, a seção passa a tratar desempenho e proteção como partes do mesmo ciclo operacional: medir, interpretar, corrigir e validar.

A figura “Arquitetura mínima de observabilidade” mostra como organizar sinais do cluster em três canais: métricas, logs e alertas, com dashboards como ponto de consumo. Note que a observabilidade não é “ferramenta”, mas um arranjo: coleta, armazenamento, visualização e processo de resposta.

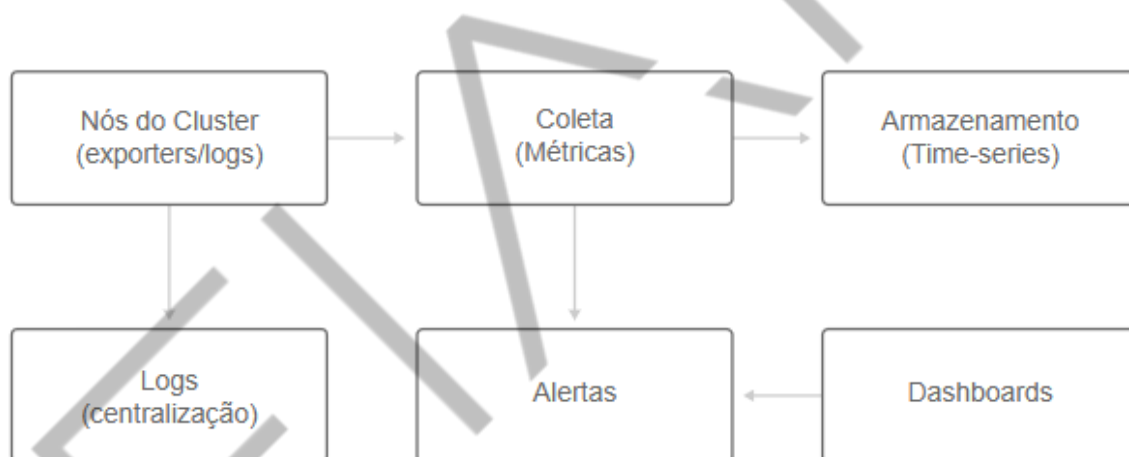


Figura 16 - Arquitetura mínima de observabilidade
Fonte: Elaborado pelo autor (2026)

Uma vez que o cluster passa a ser acompanhado por logs e métricas, o diagnóstico deixa de ser um “jogo de adivinhação” e vira um processo. Isso é crucial porque, em ambientes distribuídos, sintomas parecidos podem ter origens completamente diferentes: a mesma lentidão percebida por um usuário pode ser causada por saturação de CPU em um nó, contenção de Entrada/Saída (Input/Output – I/O) no armazenamento, fila de rede, falta de memória ou até instabilidade intermitente entre serviços. Por isso, a orientação central é simples: evitar suposições e trabalhar com evidências.

A figura “Ciclo de diagnóstico de desempenho” mostra um ciclo de diagnóstico repetível: do sintoma até a ação e a validação. O leitor deve notar que a etapa de teste

controlado é a que mais reduz erros: comparar antes/depois, ou comparar nós similares, ajuda a separar causa de coincidência.

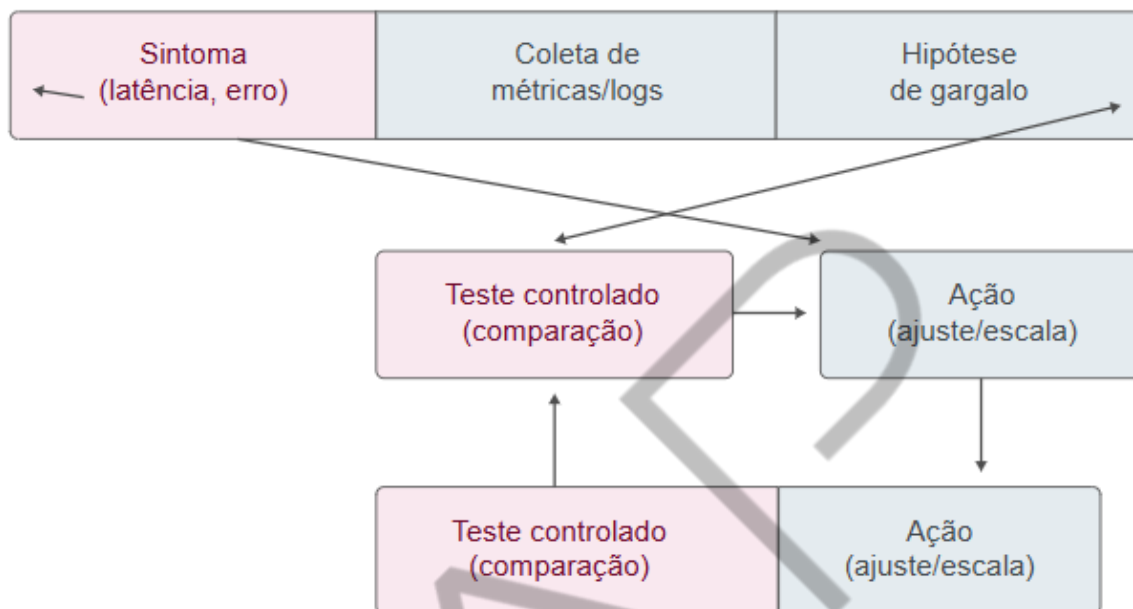


Figura 17 - Ciclo de diagnóstico de desempenho
Fonte: Elaborado pelo autor (2026)

Em vez de tentar “consertar rápido” sem entender a causa, o ciclo propõe começar pelo sintoma (por exemplo, aumento de latência, crescimento de filas ou erros intermitentes) e imediatamente passar para a coleta de sinais, métricas e logs que mostrem o que mudou e onde. Esse passo já reduz muito o espaço de hipóteses: se a CPU está estável, mas o tempo de disco disparou, a investigação deve apontar para I/O; se o tráfego de rede cresceu sem aumento proporcional de throughput, pode haver retransmissões ou gargalo de link.

Com os dados em mãos, o próximo passo é formular uma hipótese de gargalo. Aqui, a palavra-chave é hipótese, não conclusão. A figura indica que essa hipótese precisa ser colocada à prova por meio de um teste controlado (comparação). Em prática, isso significa comparar com um período “normal”, isolar uma variável ou aplicar uma mudança pequena e reversível. Por exemplo: mover temporariamente parte da carga para outro nó, ajustar um limite de concorrência, testar com um conjunto menor de dados ou reproduzir a condição em um horário controlado. O teste reduz o risco de confundir correlação com causa.

Só então vem a etapa de ação, que pode ser ajuste fino (configuração, limites, priorização), redistribuição de carga ou ampliação de capacidade (escala). A figura

também sugere a etapa que costuma ser esquecida: reavaliar. Após agir, é necessário confirmar se o sintoma realmente desapareceu e se não surgiram efeitos colaterais (por exemplo, melhora de latência à custa de aumento de erro, ou redução de fila com piora de throughput). Essa volta ao início fecha o ciclo e transforma correções pontuais em aprendizado operacional.

Além de orientar equipes pequenas, esse ciclo tem valor didático: ele ensina um padrão mental replicável para lidar com complexidade. Em vez de “engenharia por palpites”, o aluno aprende a pensar em termos de sinais observáveis, hipóteses verificáveis e mudanças controladas. Ao longo do tempo, esse hábito reduz incidentes repetidos, melhora a previsibilidade e ajuda a manter o cluster saudável mesmo quando a demanda cresce ou quando novas configurações são introduzidas.

Depois de adotar o ciclo de diagnóstico, observar sintomas, coletar sinais, formular hipótese, testar e agir, surge a pergunta prática: quais sinais valem a pena acompanhar o tempo todo? Em um cluster, é fácil cair no excesso: existem dezenas de métricas possíveis por nó, por serviço e por camada de infraestrutura. O risco é monitorar “de tudo um pouco” e, no momento do incidente, não conseguir distinguir o que realmente mudou. Por isso, a recomendação operacional é trabalhar com um conjunto enxuto de indicadores que representem bem a **saúde do ambiente**.

Para manter o desempenho previsível, algumas medições são particularmente úteis porque apontam diretamente para os gargalos mais comuns. Utilização de processador (Central Processing Unit – CPU) e memória por nó ajuda a identificar saturação e competição por recursos. Métricas de armazenamento, como latência, número de operações por segundo (Input/Output Operations Per Second – IOPS) e vazão (throughput), revelam quando o sistema está preso em Entrada/Saída (I/O), mesmo que a CPU esteja “folgada”. Na rede, tráfego, retransmissões e filas mostram congestionamento e perda de pacotes, que frequentemente aparecem como lentidão intermitente e difícil de explicar sem dados.

Em ambientes com filas, como aqueles que executam jobs agendados, há um componente adicional de experiência do usuário que não aparece apenas olhando CPU ou memória. Mesmo com recursos disponíveis, uma fila mal dimensionada ou mal configurada pode crescer e aumentar tempos de espera. Nesse caso, métricas como tamanho da fila e tempo médio de espera são essenciais para interpretar por

que tarefas “demoram para começar” e para decidir se o problema é capacidade, política de prioridade ou comportamento de pico.

O quadro “Métricas essenciais para gestão de desempenho (visão geral)” organiza essas métricas por categoria e resume o que elas sinalizam. Ela funciona como um guia de leitura: quando um sintoma aparece (por exemplo, latência alta), o quadro ajuda a escolher onde olhar primeiro, evitando tentativas aleatórias. O ponto central não é coletar o máximo possível, e sim monitorar de forma consistente aquilo que realmente muda quando o cluster entra em estresse.

Categoria	Exemplos de métricas	O que indicam
CPU	Utilização, load average, throttling	Saturação de processamento e competição por CPU
Memória	Uso, swap, cache de página	Pressão de memória e risco de degradação
Disco/I-O	Latência, IOPS, throughput	Gargalos de armazenamento e contenção
Rede	Throughput, retransmissões, filas	Congestionamento, perda e latência
Serviços	Erros, tempo de resposta, disponibilidade	Experiência do usuário e estabilidade

Quadro 6 - Métricas essenciais para gestão de desempenho (visão geral)

Fonte: Elaborado pelo autor (2026)

Apesar do quadro “Métricas essenciais para gestão de desempenho (visão geral)” apresentar as métricas em categorias técnicas, na prática, essas informações precisam ser consolidadas de forma clara e visual para apoiar decisões. É nesse ponto que entra o conceito de painel de acompanhamento (*dashboard*). Enquanto a tabela organiza o “o que medir”, a figura “Dashboard de saúde do cluster (modelo conceitual)” ilustra “como visualizar” essas medições de maneira integrada, ou seja, ela apresenta um layout didático de dashboard.

Deve-se notar a separação entre métricas de recursos (CPU, memória, rede), métricas de trabalho (fila, jobs) e métricas de serviço (erros, disponibilidade). Essa organização facilita conversas entre usuários e operadores, pois conecta sintomas a causas prováveis, permitindo uma leitura rápida do estado geral do cluster.

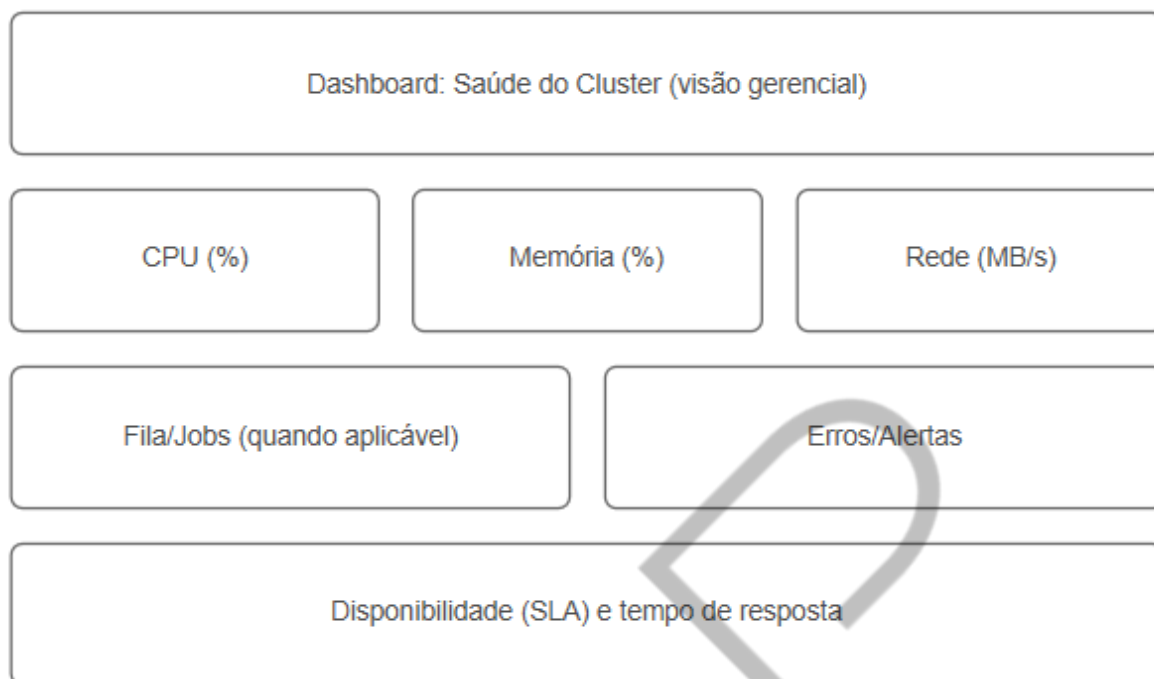


Figura 18 - Dashboard de saúde do cluster (modelo conceitual)
Fonte: Elaborado pelo autor (2026)

Mais especificamente, a figura anterior mostra uma visão gerencial simplificada, em que indicadores de processamento (CPU), memória e rede aparecem lado a lado com métricas de fila, erros e disponibilidade. Essa organização não é casual. Ela reflete a ideia discutida anteriormente: desempenho e estabilidade devem ser analisados de forma conjunta. Um aumento de CPU isolado pode não ser preocupante; porém, se vier acompanhado de crescimento de fila e aumento de tempo de resposta, o cenário muda e exige investigação.

Outro aspecto importante é que o painel combina métricas de infraestrutura (como uso de CPU e memória) com métricas percebidas pelo usuário (tempo de resposta e disponibilidade associada ao Acordo de Nível de Serviço – SLA). Essa integração evita uma visão limitada à camada técnica. Um cluster pode estar “saudável” em termos de recursos, mas ainda assim entregar uma experiência insatisfatória se houver problemas na aplicação ou na organização de carga.

Portanto, a figura deve ser lida como uma tradução visual do quadro anterior: não se trata de acompanhar dezenas de gráficos complexos, mas de selecionar indicadores representativos e organizá-los de forma coerente. Um painel bem estruturado reduz o tempo para detectar anomalias, facilita a comunicação com equipes não técnicas e sustenta decisões baseadas em evidências, mantendo o cluster dentro de uma faixa operacional estável.

A segurança em clusters não começa com ferramentas complexas, mas com decisões estruturais consistentes. Reduzir a superfície de ataque, isto é, diminuir pontos expostos e serviços desnecessários, já elimina grande parte dos riscos comuns. Manter atualizações em dia, aplicar o princípio do privilégio mínimo (cada usuário ou serviço com apenas as permissões necessárias) e separar redes administrativas de redes de dados são medidas básicas que produzem impacto imediato. O mais importante é tratar essas práticas como padrão de configuração desde o início, e não como um complemento aplicado apenas após um incidente.

A figura “Defesa em profundidade: camadas de segurança” ilustra o conceito de defesa em profundidade, organizando segurança em camadas concêntricas: se uma falhar, as demais reduzem o impacto.

Defesa em profundidade

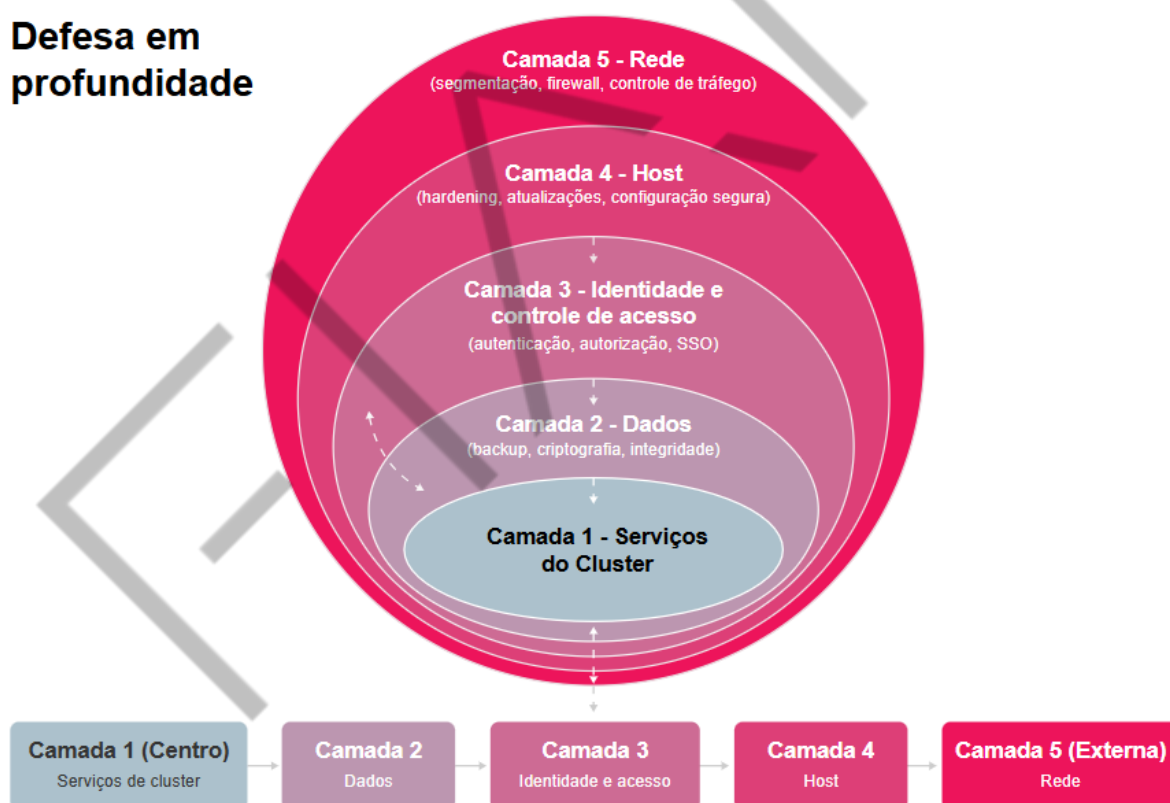


Figura 19 - Defesa em profundidade: camadas de segurança
Fonte: Elaborado pelo autor (2026)

No centro estão os serviços do cluster, aquilo que efetivamente entrega valor. Ao redor deles aparecem camadas de proteção: dados protegidos por backup e controle de acesso, identidades gerenciadas com autenticação e autorização adequadas, sistemas reforçados (*hardening*) e aplicação de correções, e, na camada mais externa, segmentação de rede e políticas de firewall. A leitura correta da figura

“Defesa em profundidade: camadas de segurança” é compreender que nenhuma camada, isoladamente, é suficiente; a proteção resulta da combinação entre elas.

Em ambientes educacionais, essa abordagem precisa equilibrar proteção e flexibilidade. O cluster deve ser protegido contra acessos indevidos e configurações indevidas, mas sem criar barreiras que impeçam a experimentação e o aprendizado. Isso significa definir limites claros (como acesso via Secure Shell com chaves, contas individuais e permissões restritas), mas permitir que o estudante explore dentro de um espaço controlado. Já em ambientes corporativos, além da proteção técnica, entram requisitos de conformidade, rastreabilidade e auditoria, ou seja, capacidade de demonstrar quem fez o quê e quando.

O conceito central reforçado pela figura “Defesa em profundidade: camadas de segurança” é que a segurança eficaz não depende de um único mecanismo, mas de camadas coordenadas. Se uma proteção falhar, por exemplo, uma credencial comprometida, outras barreiras devem limitar o impacto. Essa visão reduz o risco de falhas catastróficas e transforma a segurança em parte integrante da arquitetura do cluster, alinhada com desempenho, disponibilidade e operação contínua.

4.2 Segurança operacional e proteção de dados

Controle de acesso é um dos pontos onde a defesa em camadas vira prática do dia a dia. Em clusters, é comum começar pelo essencial: acesso remoto via Acesso Seguro ao Shell (Secure Shell – SSH) com chaves criptográficas, contas individuais e bloqueio de compartilhamento de usuários. Esse cuidado simples já reduz drasticamente riscos comuns, como senhas fracas, credenciais reutilizadas e ausência de autoria sobre ações administrativas. A ideia não é burocratizar o uso do ambiente, e sim garantir que o cluster permaneça utilizável e auditável conforme cresce.

Mesmo em ambientes pequenos, rastreabilidade é indispensável. Se ocorrer uma falha, um incidente ou uma mudança não planejada, deve ser possível responder perguntas básicas: quem acessou, quando acessou e quais ações foram executadas. À medida que o cluster passa a oferecer múltiplas interfaces, como painéis web, APIs, notebooks e ferramentas de orquestração, o controle de acesso precisa evoluir para

mecanismos mais estruturados, como Controle de Acesso Baseado em Papéis (Role-Based Access Control – RBAC) e integração com diretórios corporativos. O ponto didático aqui é direto: acesso não significa apenas “entrar no servidor”, mas controlar permissões e ações dentro do sistema, reduzindo o impacto de erros e limitando danos em caso de comprometimento.

Backups e recuperação completam a camada de proteção do ativo mais crítico: os dados. Um cluster pode ter excelente desempenho e alta disponibilidade para serviços, mas ainda assim falhar no mundo real se não conseguir recuperar informações após eventos comuns, especialmente erro humano, exclusões acidentais, corrupção de arquivos ou atualizações malsucedidas. Por isso, a recuperação deve ser tratada como parte da arquitetura operacional, com metas claras e verificáveis.

Duas métricas ajudam a transformar “backup” em decisão concreta. O Objetivo de Ponto de Recuperação (Recovery Point Objective – RPO) define quanto dado a organização aceita perder, isto é, qual é a “janela máxima” de perda. Já o Objetivo de Tempo de Recuperação (Recovery Time Objective – RTO) define quanto tempo o serviço pode ficar indisponível até voltar ao funcionamento aceitável. Em termos práticos, RPO influencia a frequência e o tipo de cópia; RTO influencia a estratégia de restauração, a automação e a preparação do ambiente para retornar rapidamente.

Termo	Pergunta que responde	Exemplo de decisão
RPO	Quanto dado posso perder?	Backup a cada 24h <i>versus</i> a cada 1h.
RTO	Quanto tempo posso ficar fora?	Restaurar em horas <i>versus</i> em minutos.

Quadro 7 - RPO e RTO (interpretação prática)
Fonte: Elaborado pelo autor (2026)

A incorporação de práticas de monitoramento e segurança torna-se mais clara quando observada em cenários concretos. Em vez de tratar desempenho e proteção como conceitos abstratos, os exemplos a seguir mostram como decisões simples, porém estruturadas, produzem impacto real na estabilidade de um cluster. Ambos refletem práticas amplamente adotadas no mercado, adaptadas para contextos educacionais e corporativos.

- **Exemplo prático 1:** considere um laboratório acadêmico que não dispõe de equipe dedicada de operações. Em vez de implementar uma plataforma complexa de observabilidade, o grupo decide adotar um modelo mínimo e

sustentável de monitoramento. O painel acompanha, por nó, quatro indicadores essenciais: uso de CPU, memória, disco e rede. Além disso, são configurados alertas simples para eventos críticos, como disco cheio, nó indisponível ou uso excessivo de memória virtual (swap). O objetivo não é analisar centenas de métricas, mas identificar rapidamente comportamentos anormais antes que afetem uma aula ou uma entrega de projeto. Essa abordagem espelha práticas utilizadas com ferramentas como Prometheus e Grafana em ambientes reais, mas com escopo ajustado à maturidade da equipe.

A figura “Painel mínimo de monitoramento por nó” ilustra uma visualização simplificada contendo CPU, memória, disco e rede por nó, com alertas básicos configurados. A leitura rápida permite detectar desvios antes que se tornem indisponibilidade percebida.



Figura 20 - Painel mínimo de monitoramento por nó
Fonte: Elaborado pelo autor (2026)

- **Exemplo prático 2:** o contexto é corporativo: o cluster hospeda serviços internos críticos. Nesse ambiente, a preocupação não é apenas desempenho, mas também exposição a riscos operacionais. O time estabelece um conjunto de medidas fundamentais: atualizações periódicas do sistema, redução de portas abertas ao mínimo necessário, autenticação

por chaves SSH, revisão regular de usuários e grupos, e testes periódicos de restauração de backup. Essas ações não exigem tecnologias sofisticadas, mas criam uma base sólida de proteção. O ganho esperado é duplo: diminuir a probabilidade de incidentes simples (como exploração de serviço desatualizado) e aumentar a capacidade de resposta quando algo inesperado ocorre.

A figura a seguir mostra uma representação das medidas estruturais: atualização contínua, controle de acesso por chaves, redução de superfície de ataque, auditoria de usuários e estratégia de backup testado.



Figura 21 - Camadas básicas de reforço de segurança e auditoria em cluster corporativo
Fonte: Elaborado pelo autor (2026)

Esses dois exemplos reforçam um princípio comum ao longo do capítulo: maturidade operacional não significa complexidade excessiva, mas coerência entre objetivos, recursos disponíveis e práticas adotadas. Seja em laboratório ou em empresa, desempenho e segurança dependem de monitoramento consistente, configuração disciplinada e revisão periódica, pilares que sustentam a estabilidade de qualquer cluster moderno.

Área	Controle	Objetivo
Rede	Segmentação e firewall	Reduzir exposição e movimento lateral.
Acesso	Chaves SSH e contas individuais	Identidade e rastreabilidade.
Sistema	Atualizações e hardening	Reduzir vulnerabilidades.
Dados	Backup e teste de restauração	Recuperação de incidentes.
Operação	Logs e auditoria	Investigar e aprender com eventos.

Quadro 8 - Controles básicos de segurança (checklist orientado a prática)

Fonte: Elaborado pelo autor (2026)

Resumo da seção (pontos-chave):

- Observabilidade (métricas/logs) é a base para entender e melhorar o cluster em operação;
- Diagnóstico de gargalos deve seguir um ciclo: sintoma → dados → hipótese → teste → ação;
- Métricas essenciais cobrem CPU, memória, I/O, rede e indicadores de serviço;
- A segurança depende de princípios: superfície mínima, privilégio mínimo e segmentação;
- O backup e recuperação com RPO/RTO definidos são indispensáveis, mesmo em ambientes simples.

Checklist prático (manter o cluster saudável):

- Tenho painel de saúde com poucas métricas essenciais por nó e por serviço?
- Alertas estão configurados para eventos críticos (nó fora, disco cheio, erros)?
- Existe rotina de atualização e hardening, com janelas e validação?
- Acessos são individuais e auditáveis (evitar contas compartilhadas)?
- Backups são feitos e testados (restauração) com RPO/RTO conhecidos?

REFERÊNCIAS

BEYER, Betsy *et al.* **Site Reliability Engineering: How Google Runs Production Systems**. Sebastopol: O'Reilly Media, 2016. Disponível em: <<https://books.google.com.br/books?id=81UrjwEACAAJ>>. Acesso em: 19 fev. 2026.

BURNETT, Carl Alexander. **Flynn's Taxonomy of a MIMD design**. [Imagem]. 11 jun. 2007. Disponível em: <<https://commons.wikimedia.org/wiki/File:MIMD.svg>>. Acesso em: 13 out. 2025.

BURNETT, Carl Alexander. **Flynn's Taxonomy of a SIMD design**. [Imagem]. 11 jun. 2007. Disponível em: <<https://commons.wikimedia.org/wiki/File:SIMD.svg>>. Acesso em: 13 out. 2025.

DANIEL. **Gráfico ilustrando a Lei de Amdahl**. [Imagem]. 13 abr. 2008. Disponível em: <<https://commons.wikimedia.org/wiki/File:AmdahlsLaw.svg>>. Acesso em: 13 out. 2025.

GRAFANA LABS. **Dashboards — Overview**. [S. l.], [s. d.]. Disponível em: <<https://grafana.com/docs/grafana/latest/fundamentals/dashboards-overview/>>. Acesso em: 19 fev. 2026.

KLEPPMANN, Martin. **Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems**. Sebastopol: O'Reilly Media, 2017. Disponível em: <<https://books.google.com.br/books?id=p1heDgAAQBAJ>>. Acesso em: 19 fev. 2026.

NADAS, S. (ed.). **Virtual Router Redundancy Protocol (VRRP) Version 3 for IPv4 and IPv6** [RFC 5798]. IETF, mar. 2010. Disponível em: <<https://www.rfc-editor.org/rfc/rfc5798.html>>. Acesso em: 19 fev. 2026.

NGINX. **HTTP Load Balancing**. [S. l.], [s. d.]. Disponível em: <<https://docs.nginx.com/nginx/admin-guide/load-balancer/http-load-balancer/>>. Acesso em: 19 fev. 2026.

ORACLE. **Installing and Configuring Pacemaker and Corosync**. Oracle Linux 8: High Availability, [s. d.]. Disponível em: <<https://docs.oracle.com/en/operating-systems/oracle-linux/8/availability/availability-InstallingandConfiguringPacemakerandCorosync.html>>. Acesso em: 19 fev. 2026.

PROMETHEUS AUTHORS. **Prometheus: Overview**. [S. l.], [s. d.]. Disponível em: <<https://prometheus.io/docs/introduction/overview/>>. Acesso em: 19 fev. 2026.

SCARFONE, Karen; JANSEN, Wayne; TRACY, Miles. **Guide to General Server Security**. Gaithersburg: National Institute of Standards and Technology (NIST), 2008.

(NIST Special Publication 800-123). Disponível em:
<<https://csrc.nist.gov/pubs/sp/800/123/final>>. Acesso em: 19 fev. 2026.

EMASP